

H A N D B O O K

Typst

From ZERO to HERO

Stefano Coelati Rama

Scritto ed impaginato interamente in Typst

Typst: from ZERO to HERO

Manuale (quasi) Completo in Italiano

Questo documento è stato scritto *interamente* nel linguaggio Typst, senza alcun ausilio di altri strumenti. Costituisce esso stesso una dimostrazione delle capacità di impaginazione e programmazione del linguaggio. Potrebbe contenere un sacco di refusi (linguistici e non)...appreziate l'impegno.

Versione 1.0 — Giugno 2026

Tutti gli esempi presenti in questo manuale sono direttamente eseguibili nell'editor online ufficiale:

typst.app

Il contenuto di questo manuale è rilasciato per uso didattico. In caso questo manuale diventi vecchio o non si trovino alcune informazioni, vi invito ad andare nella [community](#), o nei [docs](#) per cercare informazioni. Altrimenti, come tutti i comuni mortali, chiedere all'intelligenza artificiale (ho provato ad usarla anche io - claude in particolare - per la stesura del manuale. Mi ha aiutato con qualche box, ma (forse perchè typst è troppo nuovo) il codice è quasi sempre corrotto e va modificato a mano, ergo: studiate). Questo testo non è per fini di lucro, Typst è un progetto open-source! <https://github.com/typst/typst>

I miei contatti li trovate qui: <https://stefanocoelatirama.com>

Indice

1. Prefazione	i
1.1. A chi è rivolto	i
1.2. Come usare questo manuale	i
1.3. Mie convenzioni tipografiche	ii
2. Cos'è Typst e perché usarlo	3
2.1. Markup: cos'è e come funziona	3
2.2. Confronto con gli strumenti più comuni	3
2.2.1. Typst vs Microsoft Word / LibreOffice	3
2.2.2. Typst vs LaTeX	4
2.3. Come funziona Typst: il flusso di lavoro	4
2.4. Un brevissimo cenno storico	4
3. Installazione e primo documento	5
3.1. Opzione 1: Editor online (consigliata per iniziare)	5
3.2. Opzione 2: Installazione locale	6
3.2.1. Installazione su Windows	6
3.2.2. Installazione su macOS	6
3.2.3. Installazione su Linux	6
3.2.4. Estensione VS Code	6
3.2.5. Compilare dalla riga di comando	6
3.3. Struttura di un file .typ	7
3.4. Il tuo primo documento: «Hello, W...Typst!»	7
4. Testo e formattazione di base	8
4.1. Scrivere testo normale	8
4.1.1. Paragrafi	8
4.1.2. Andare a capo senza nuovo paragrafo	9
4.2. Grassetto, corsivo e sottolineato	9
4.2.1. Grassetto	9
4.2.2. Corsivo	10
4.2.3. Combinare grassetto e corsivo	10
4.2.4. Testo cancellato (strikethrough)	10
4.2.5. Codice inline	11
4.2.6. Apice e pedice	11
4.3. Caratteri speciali e come usarli	11
4.4. Virgolette tipografiche	12
4.5. Spazi speciali	12
4.6. Linea orizzontale	13
5. Titoli e struttura del documento	13

5.1. I livelli di titolo	13
5.2. Numerazione automatica	14
5.3. L'indice automatico con <code>#outline()</code>	15
5.4. Titoli fuori dall'indice	15
5.5. Etichette e riferimenti incrociati	15
5.5.1. Assegnare un'etichetta	15
5.5.2. Fare riferimento	15
5.6. Interruzione di pagina	16
6. Liste	16
6.1. Lista puntata	16
6.1.1. Marcatori personalizzati	17
6.2. Lista numerata	17
6.2.1. Stile di numerazione	17
6.2.2. Iniziare da un numero specifico	18
6.3. Liste annidate	18
6.4. Lista di definizioni (termine: definizione)	19
6.5. Voci su più righe	19
6.6. Spaziatura tra le voci	20
7. Link e riferimenti	20
7.1. Link a pagine web	20
7.1.1. Sintassi base	20
7.1.2. URL automatici	20
7.1.3. Link senza testo visibile (solo URL)	21
7.1.4. Link a indirizzi email	21
7.2. Note a piè di pagina	21
7.3. Riepilogo link e riferimenti	21
8. Immagini e figure	22
8.1. Inserire un'immagine	22
8.1.1. Formati supportati	22
8.1.2. Parametro <code>fit</code>	22
8.2. La funzione <code>#figure()</code> : figure con didascalia	23
8.2.1. Componenti di <code>#figure()</code>	23
8.2.2. Personalizzare il supplemento	23
8.2.3. Posizionamento delle figure	23
8.3. Allineamento	23
8.4. Immagini affiancate	24
8.5. Formati e ottimizzazione delle immagini	24
8.5.1. Immagini SVG per grafica vettoriale	24
8.5.2. Immagini: solo file locali (niente URL)	24
8.5.3. Testo alternativo (accessibilità)	25
8.5.4. Clip e zoom: mostrare una porzione di immagine	25
8.5.5. Immagini affiancate con didascalie individuali	25

8.5.6. Rotazione e trasformazione	26
8.5.7. Immagine come sfondo di pagina	26
8.6. Consigli pratici	26
9. Tabelle	27
9.1. La prima tabella	27
9.2. Definire le colonne	27
9.3. Intestazioni della tabella	28
9.4. Allineamento del contenuto	28
9.5. Colori alternati (zebra stripes)	29
9.6. Bordi (stroke)	29
9.7. Celle che si estendono su più colonne/righe	29
9.8. Tabelle come figure	30
10. Matematica	30
10.1. Modalità matematica	30
10.1.1. Matematica inline	30
10.1.2. Matematica a blocco (display)	31
10.2. Operatori di base	32
10.3. Potenze e pedici	32
10.4. Frazioni	33
10.5. Radici	33
10.6. Simboli speciali comuni	33
10.6.1. Lettere greche	33
10.6.2. Altri simboli frequenti	34
10.7. Sommatorie, produttorie, limiti	34
10.8. Vettori e matrici	35
10.9. Testo all'interno delle formule	35
10.10. Formule numerate	35
10.11. Spazi in matematica	36
11. Blocchi di codice	36
11.1. Codice inline	36
11.2. Blocco di codice	36
11.3. Linguaggi supportati	37
11.4. Numeri di riga	37
11.5. Stile personalizzato del codice	37
11.6. Numero di riga e evidenziazione manuale	38
12. Introduzione al codice: il simbolo <code>\#</code>	38
12.1. Due modalità: Markup e Codice	38
12.2. Il simbolo <code>#</code> in azione	39
12.3. Espressioni inline con <code>#</code>	39
12.4. Blocchi di codice con <code>{ }</code>	39
12.5. Blocchi di contenuto con <code>[]</code>	40
12.6. La differenza fondamentale: <code>{ }</code> vs <code>[]</code>	40

12.7. Il principio di base: <code>#</code> per tutto il codice	41
12.8. Il contesto di esecuzione	41
13. Tipi di dato	42
13.1. Panoramica dei tipi	42
13.2. Stringhe	42
13.2.1. Operazioni sulle stringhe	43
13.2.2. Interpolazione di stringhe	43
13.3. Numeri interi (int)	43
13.4. Numeri decimali (float)	44
13.5. Booleani (bool)	44
13.6. Lunghezze e misure	45
13.7. Colori	45
13.8. Array (liste)	46
13.9. Dizionari (dict)	47
13.10. Content	47
13.11. none e auto	48
13.12. Scoprire il tipo di un valore	48
13.13. Conversioni tra tipi	48
14. Variabili	49
14.1. Definire una variabile con <code>#let</code>	49
14.2. Usare una variabile	49
14.3. Perché usare le variabili?	49
14.4. Scope delle variabili	50
14.5. Variabili che contengono contenuto	50
14.6. Costanti vs variabili	51
14.7. Operazioni e calcoli	51
15. Condizioni: if, else if, else	51
15.1. La struttura if di base	52
15.2. if ... else	52
15.3. if ... else if ... else	53
15.4. Operatori di confronto	53
15.5. Operatori logici	54
15.6. L'if come espressione	54
16. Cicli: for e while	55
16.1. Il ciclo for	55
16.1.1. Iterare su un array	55
16.1.2. Iterare con range	55
16.1.3. range con step	56
16.1.4. Generare una tabella con for	56
16.1.5. Iterare con indice	57
16.1.6. Il for restituisce un array	57
16.2. Il ciclo while	57

16.3. break e continue	58
16.4. Generare contenuto complesso con cicli	58
17. Funzioni	59
17.1. Funzioni predefinite di Typst	59
17.2. Definire le proprie funzioni	60
17.2.1. Funzione semplice	60
17.2.2. Funzione con corpo complesso	60
17.3. Parametri con nome e valori predefiniti	60
17.4. Funzioni che accettano contenuto	61
17.4.1. La sintassi speciale con parametro body	62
17.5. Funzioni che restituiscono valori	63
17.6. Funzioni freccia (arrow functions / closures)	63
17.7. Ricorsione	63
17.8. Esempio completo: template di carta	64
18. Le regole set	65
18.1. Cos'è una regola set?	65
18.2. set text — il testo	65
18.3. set par — i paragrafi	66
18.4. set heading — i titoli	67
18.5. set page — la pagina	67
18.6. set list e set enum	68
18.7. set table — le tabelle	68
18.8. set figure — le figure	68
18.9. set math.equation — le equazioni	68
18.10. Lo scope delle regole set	69
18.11. set in funzioni	69
19. Le regole show	70
19.1. La differenza tra set e show	70
19.2. Trasformare il grassetto	70
19.3. Trasformare i link	70
19.4. Trasformare il codice inline	71
19.5. Selettori: .where()	71
19.6. Trasformare i titoli	72
19.6.1. Prima sezione viola	72
19.6.2. Seconda sezione viola	72
19.7. Trasformare il documento intero (show globale)	72
19.8. Trasformare le figure	73
19.9. show per sostituzione testo	73
20. Colori e grafica	74
20.1. Sistemi di colore	74
20.1.1. RGB	74
20.1.2. Scala di grigi	74

20.1.3. OKLCH (moderno, perceptually uniform)	74
20.1.4. Colori predefiniti	75
20.2. Manipolare i colori	75
20.3. Gradienti	76
20.3.1. Gradiente lineare	76
20.3.2. Gradiente radiale	76
20.4. Disegnare forme base	77
20.4.1. Linea	77
20.4.2. Rettangolo	77
20.4.3. Cerchio ed ellisse	77
20.5. Lo stroke	78
21. Box e blocchi	79
21.1. box vs block: la differenza	79
21.2. Parametri di box e block	79
21.2.1. Angoli arrotondati personalizzati	80
21.2.2. Inset dettagliato	80
21.3. Impedire le interruzioni di pagina	80
21.4. Posizionamento con place()	81
21.5. Stack: impilare elementi	81
21.6. Allineamento	82
22. Impaginazione avanzata	83
22.1. Intestazioni e piè di pagina	83
22.1.1. Intestazione semplice	83
22.1.2. Intestazione con numero di pagina	83
22.1.3. Il contesto (context) e la consapevolezza del documento	83
22.1.4. Intestazione con titolo sezione corrente	84
22.2. Numerazione pagine	84
22.2.1. Cambiare numerazione nel documento	84
22.3. Colonne	85
22.3.1. Saltare una colonna	85
22.4. Pagine speciali	86
22.4.1. Pagina con margini diversi	86
22.4.2. Pagina in landscape (orizzontale)	86
23. Layout a griglia con grid()	86
23.1. Sintassi di base	86
23.2. Definire le colonne	87
23.3. Esempi pratici di layout	87
23.3.1. Layout copertina (icona + testo)	87
23.3.2. Layout a tre colonne informative	88
23.3.3. Grid con celle che si estendono	88
23.4. La funzione layout()	88
24. File multipli e moduli	89

24.1.	#include — includere contenuto	89
24.1.1.	Vantaggi di #include	89
24.2.	#import — importare funzioni e variabili	89
24.2.1.	Importare tutto con *	90
24.2.2.	Importare con alias	90
24.2.3.	Differenza tra import e include	90
24.3.	Organizzare un progetto grande	90
24.4.	Variabili condivise tra file	91
25.	Pacchetti	92
25.1.	Dove trovare i pacchetti	92
25.2.	Come importare un pacchetto	92
25.3.	Pacchetti principali	92
25.3.1.	showybox — Box stilizzate	92
25.3.2.	codly — Codice con numeri di riga	93
25.3.3.	cetz — Disegno vettoriale	93
25.3.4.	fletcher — Diagrammi e frecce	93
25.3.5.	physica — Notazione fisica	93
25.3.6.	polylux — Presentazioni	93
25.3.7.	tablex — Tabelle avanzate	94
25.3.8.	lovelace — Pseudocodice e algoritmi	94
25.3.9.	codelst — Liste di codice con riferimenti	94
25.3.10.	diagraph — Grafi con GraphViz	94
25.4.	Come scegliere la versione giusta	94
26.	Documento accademico completo	95
26.1.	Template minimo per una tesi	95
26.2.	Equazioni numerate e riferimenti	97
26.3.	Citazioni bibliografiche	98
26.4.	Figure e tabelle accademiche	98
27.	Template Curriculum Vitae	99
27.1.	Template CV a due colonne	99
27.2.	Variante CV minimalista (una colonna)	103
28.	Presentazioni (Slide)	103
28.1.	Setup con Polylux	103
28.2.	Rivelazione progressiva (animazioni)	106
28.3.	Il pacchetto Touying (alternativa avanzata)	106
29.	Argomenti avanzati	107
29.1.	Il sistema context	107
29.1.1.	Contatori	107
29.1.2.	Query	108
29.1.3.	State (stato globale)	108
29.2.	Algoritmi di testo avanzati	108
29.2.1.	Testo con line-by-line control	108

29.2.2. Testo verticale	109
29.3. Pattern e ripetizioni	109
29.4. Disegno avanzato con CeTZ	110
29.5. Diagrammi con Fletcher	110
Appendice A – Errori Comuni	111
29.6. Errori di sintassi frequenti	111
29.7. Messaggi di errore comuni	111
Appendice B – Risorse e Community	112
29.8. Documentazione ufficiale	112
29.9. Community e supporto	112
29.10. Strumenti complementari	112
Appendice C – Font e Tipografia	112
29.11. Come specificare i font	112
29.12. Font disponibili nell’editor online	113
29.13. Pesi e stili del font	113
29.14. Misure tipografiche	114
29.15. Highlight e enfasi senza grassetto	114
Appendice D – Contatori e Numerazione	115
29.16. Contatori predefiniti	115
29.17. Creare un contatore personalizzato	115
29.18. Resetare un contatore	116
29.19. Visualizzare il numero totale di pagine	116
Appendice E – Template Pronti all’Uso	117
29.20. Template: Documento generico professionale	117
29.21. Template: Documento minimalista	117
29.22. Template: Documento tecnico (stile manuale)	117
29.23. Template: Dispensa universitaria	118
29.24. Template: Newsletter / Bollettino	119
29.25. Template: Poster accademico (A1)	119
Appendice F (Parte I) – Note Aggiuntive: Slide e Presentazioni	120
29.26. Panoramica degli strumenti	120
29.26.1. Beamer	120
29.26.2. R Markdown	121
29.26.3. Quarto	122
29.26.4. knitr	123
29.26.5. Polylux	124
29.26.6. Touying	125
29.27. Confronto finale: quale scegliere?	127
Appendice F (Parte II) – Note Aggiuntive: Programmazione e Automazione	127
29.28. Il pacchetto Python <code>typst</code>	127
29.28.1. Compilare un file	127

29.28.2. Passare parametri con sys.inputs	128
29.29. Leggere dati JSON nel documento	129
29.30. Il pattern Dati → Template → PDF	130
29.31. Produzione in batch: esempio certificati	130
29.32. Integrazione con FastAPI	131
29.33. GitHub Actions: report automatici	132
29.34. Buone pratiche per Typst in produzione	133

1. Prefazione

Questo manuale nasce da una domanda semplice: **è possibile imparare Typst partendo da zero assoluto? (Anche senza aver mai programmato?)** La risposta è sì, e questo testo ve lo dimostrerà pagina dopo pagina.

Typst è un linguaggio moderno per creare documenti di qualità professionale: articoli scientifici, tesi di laurea, presentazioni, CV, libri, dispense. È più semplice di LaTeX, più potente di Word, versatile quanto markdown ma, mi permetto di dire, molto più elegante di tutti questi.

1.1. A chi è rivolto

Questo manuale è pensato per:

- **Studenti e ricercatori** che vogliono produrre tesi e articoli impeccabili senza impazzire con la formattazione
- **Professionisti** che vogliono documenti belli senza dipendere da Word
- **Curiosi** che amano capire come funzionano le cose
- **Chiunque non abbia mai programmato**, partiamo davvero da zero

Non è richiesta nessuna conoscenza di programmazione, LaTeX, o HTML. Basta saper scrivere.

1.2. Come usare questo manuale

Il manuale è diviso in sette parti, in ordine crescente di complessità:

Parte I – Fondamenti (Cap. 1–5): Cos'è Typst, come si installa, come si scrive testo semplice, grassetto, corsivo, titoli, liste.

Parte II – Elementi aggiuntivi (Cap. 6–10): Link, immagini, tabelle, formule matematiche, blocchi di codice.

Parte III – Il Linguaggio (Cap. 11–16): Il sistema di scripting: variabili, tipi, condizioni, cicli, funzioni.

Parte IV – La Stilizzazione (Cap. 17–19): Le regole `set` e `show`. Come personalizzare ogni aspetto del documento.

Parte V – Un po' di Layout Avanzato (Cap. 20–23): Colori, box, impaginazione professionale, griglie e colonne.

Parte VI – Organizzazione del Progetto (Cap. 24–25): File multipli, moduli, pacchetti della comunità.

Parte VII – Una parte di progetti Reali (Cap. 26–28): Template completi: documento accademico, CV, presentazione.

Consiglio

Apri typst.app sul browser mentre leggi. Ogni volta che vedi un esempio di codice, copialo e incollalo nell'editor: vedrai il risultato in tempo reale. Se impari facendo in modo pratico, impari prima e sei anche più veloce.

1.3. Mie convenzioni tipografiche

Nel testo troverai questi elementi ricorrenti:

codice	Codice Typst, nomi di funzioni, file
Nota	Informazione utile da tenere a mente
Attenzione	Errori comuni da evitare
Consiglio	Trucchi e buone pratiche
Prova tu!	Esercizio pratico da eseguire ¹

¹ Alla fine non li ho usati...ma mi piaceva comunque l'idea di avere questa tag, quindi eccola qui.

2. Cos'è Typst e perché usarlo

Typst è un linguaggio di *markup* e di *programmazione* progettato per creare documenti di alta qualità tipografica. In parole semplici: scrivi del testo in un file, aggiungi alcune istruzioni speciali, e Typst produce automaticamente un PDF professionale.

Pensa a Typst come a qualcosa che si trova a metà strada tra Microsoft Word e LaTeX.

2.1. Markup: cos'è e come funziona

Prima di tutto, chiariamo il termine **markup**. Un linguaggio di markup è un sistema in cui scrivi il tuo contenuto (testo, immagini, equazioni...) insieme a delle *marcature*, delle istruzioni speciali che descrivono come quel contenuto deve apparire.

Il principio è **separare il contenuto dalla forma**.

- In Word, clicchi su «Grassetto» e il testo diventa grassetto.
- In Typst, come in latex, scrivi `*testo in grassetto*` e Typst lo rende grassetto quando compila il PDF.

Questo approccio ha vantaggi enormi:

- Il documento è un semplice file di testo: leggero e portabile.
- La formattazione è consistente in tutto il documento
- Puoi concentrarti sul contenuto mentre scrivi, senza distrarti con i menu di formattazione
- Puoi automatizzare qualsiasi cosa: numeri di pagina, indici, riferimenti incrociati, tabelle generare dal codice. Non devi metterti le mani nei capelli (se ne hai...fortunato tu...) se vuoi solamente modificare un pezzo dell'indice o l'ordine dei paragrafi, fa tutto typst in automatico.

2.2. Confronto con gli strumenti più comuni

2.2.1. Typst vs Microsoft Word / LibreOffice

Aspetto	Word / LibreOffice	Typst
Tipo	WYSIWYG (vedi subito il risultato)	Markup (compili per vedere il risultato)
Curva di apprendimento	Bassa all'inizio	Bassa — più semplice di LaTeX
Consistenza	Difficile da mantenere	Perfetta automaticamente
Formule matematiche	Difficili	Eccellenti, semplici

File	Formato binario .docx	Testo puro .typ
Velocità	Lenta su documenti grandi	Compilazione istantanea
Automazione	Macro VBA (complesse)	Scripting integrato (semplice)
Costo	Costoso / gratuito	Gratuito e open-source

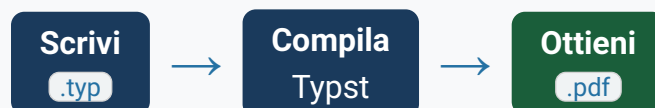
2.2.2. Typst vs LaTeX

LaTeX è il re incontrastato della tipografia accademica da decenni. Typst ne eredita l'eccellenza tipografica ma risolve i suoi «problemi» storici:

Aspetto	LaTeX	Typst
Errori	Messaggi criptici da linguaggio di programmazione	Messaggi molto chiari e precisi
Compilazione	Lenta (secondi/minuti)	Istantanea e veloce anche su file enormi
Sintassi	Complicata e un po' vecchia	Coincisa e molto nuova
Imparare	Ripida per un principiante	Molto accessibile
Pacchetti	Migliaia (CTAN)	In crescita (Universe)
Qualità tipografica	Eccellente	Eccellente
Matematica	Ottima	Ottima, ma sintassi più moderna

2.3. Come funziona Typst: il flusso di lavoro

Il processo è sempre lo stesso:



Nell'editor online (typst.app), la compilazione avviene automaticamente mentre scrivi: ogni volta che fai una pausa di scrittura, il PDF si aggiorna in tempo reale nel pannello di destra. È sempre istantaneo (È istantaneo anche se avete un file enorme, come in questo caso, l'importante è scrivere/modificare pezzi di codice e non fare copia-incolla tutto insieme. In quel caso ci potrebbe volere qualche secondo. (Non vibecodate...)).

2.4. Un brevissimo cenno storico

Typst è nato come progetto di tesi magistrale di due studenti dell'Università Tecnica di Berlino (TU Berlin), Laurenz Mädje e Martin Haug. Il progetto ha aperto la sua beta pubblica

nel marzo 2023, e da allora ha raccolto una comunità entusiasta in tutto il mondo. È open-source (licenza Apache 2.0) e disponibile gratuitamente sia come applicazione web che come strumento da riga di comando.

Nota

In questo manuale usiamo la versione online di Typst (typst.app) per tutti gli esempi. Non è necessario installare nulla sul proprio computer per iniziare. In caso vogliate, potete installarlo sul vostro dispositivo e scrivere con ciò che volete (lo questo documento l'ho scritto in LazyVIM).

3. Installazione e primo documento

3.1. Opzione 1: Editor online (consigliata per iniziare)

Il modo più semplice per iniziare è l'editor online ufficiale. Non richiede installazioni, funziona su qualsiasi browser.

1. Apri il browser e vai su typst.app
2. Crea un account gratuito (o entra con Google/GitHub)
3. Clicca su «**Empty document**» (o «**Start from template**» se volete già qualcosa di pronto)
4. Si apre l'editor: a sinistra il codice, a destra il PDF in anteprima

L'editor online ha queste caratteristiche:

- Anteprima PDF in tempo reale
- Evidenziazione della sintassi
- Autocompletamento
- Messaggi di errore istantanei
- Possibilità di esportare il PDF (e non solo)
- Archiviazione dei progetti nel cloud
- Impostazione VIM della tastiera (consigliata)²

Consiglio

Se sei alle prime armi, ti consiglio, soprattutto per i primi capitoli di questo manuale, di usare esclusivamente l'editor online. È il modo più rapido per sperimentare senza distrazioni, è semplicissimo e non devi fare altro che scrivere.

²Visto che si parla di tastiera, mi sembra giusto dirvi che se usate la versione online, come vi consiglio, ci sono tantissime scorciatoie da tastiera. Scopritene ed usatene quanto potete, vi velocizzano di molto il lavoro!

3.2. Opzione 2: Installazione locale

Per chi vuole lavorare offline o integrare Typst nel proprio flusso di lavoro con un editor di testo come VS Code (o VIM!!).

3.2.1. Installazione su Windows

```
winget install --id Typst.Typst
```

Oppure semplicemente scarica il file `.exe` dalla pagina delle release: <https://github.com/typst/typst/releases>

3.2.2. Installazione su macOS

Con Homebrew:

```
brew install typst
```

3.2.3. Installazione su Linux

Con il gestore pacchetti del sistema (♥ Arch ♥³, NixOS, ecc.) o tramite:

```
cargo install --git https://github.com/typst/typst
```

Oppure scarica il binario pre-compilato dalla pagina release.

3.2.4. Estensione VS Code

Per quanto non mi piaccia VS so che tanti lo usano, quindi se anche tu usi Visual Studio Code (e dunque sei una brutta persona), installa l'estensione **Tinymist Typst** dall'Extensions Marketplace. Fornisce:

- Anteprima PDF integrata
- Autocompletamento e suggerimenti
- Navigazione nel documento
- Evidenziazione errori

3.2.5. Compilare dalla riga di comando

Una volta installato, compila un file con:

```
typst compile mio-documento.typ
```

Per la modalità watch (ricompila automaticamente a ogni salvataggio):

³Amo archlinux, dovete amare arch linux anche voi. In questo handbook non le ho inserite, ma se volete anche voi queste bellissime emoji, trovate tutti i codici a questo indirizzo: <https://typst.app/docs/reference/symbols/emoji/>

```
typst watch mio-documento.typ
```

3.3. Struttura di un file .typ

Un file Typst ha estensione `.typ` ed è un semplice file di testo. Non c'è una struttura obbligatoria: puoi iniziare a scrivere direttamente.

Un file tipico è composto da:

```
// Impostazioni del documento (opzionali ma consigliate)
#set document(title: "Il mio documento", author: "Stefano Coelati Rama")
#set page(paper: "a4", margin: 2cm)
#set text(font: "Linux Libertine", size: 11pt)

// Da qui in poi: il contenuto del documento
= Titolo del documento

Qui scrivo il mio testo normale.

== Una sezione

Altro testo...
```

Le righe che iniziano con `//` sono **commenti**: Typst le ignora completamente. Sono utili per lasciare note a te stesso.

3.4. Il tuo primo documento: «Hello, W...Typst!»

Apri l'editor online, cancella tutto quello che c'è (se c'è), e scrivi questo:

► Esempio

= Ciao, Typst!

Questo è il mio primo documento scritto in Typst.
È molto semplice iniziare!

== Una sezione

Posso avere ***grassetto***, *_corsivo_* e anche ``codice``.

- Prima voce

- Seconda voce
- Terza voce

Compila (nell'editor online avviene in automatico) e vedrai il PDF apparire a destra.

Prova tu!

Nell'editor online:

1. Copia il codice qui sopra
2. Modifica il titolo con il tuo nome
3. Aggiungi un'altra sezione con qualche testo
4. Aggiungi una lista con i tuoi tre cibi preferiti

Nota

In questo manuale gli esempi di codice sono mostrati con il linguaggio `typ` evidenziato. Quando copi il codice, copia **solo** il contenuto, non i tre backtick iniziali e finali (tre backtick: ```,```,```) che sono solo la marcatura del blocco di codice nel manuale (è obbligatoria altrimenti da errore, vedrete questi «tag» spesso anche nelle sezioni).

4. Testo e formattazione di base

Questo è il capitolo più pratico: qui impari a fare tutto quello che fai normalmente in un editor di testo, ma in Typst.

4.1. Scrivere testo normale

Per scrivere testo, scrivi e basta. Niente di speciale. Typst considera tutto ciò che non inizia con un carattere speciale come testo normale.

Questo è testo normale.
Puoi scrivere tutto quello che vuoi.
Numeri: 42, 3.14, -7.
Punteggiatura: virgole, punti, punti esclamativi!

4.1.1. Paragrafi

Per creare un nuovo paragrafo, lascia **una riga vuota** tra due blocchi di testo:

► Paragrafi

CODICE TYPST

Questo è il primo paragrafo.
Questa riga continua nello stesso paragrafo.

Questo è un secondo paragrafo, separato da una riga vuota.

RISULTATO

Questo è il primo paragrafo. Questa riga continua nello stesso paragrafo.

Questo è un secondo paragrafo, separato da una riga vuota.

Nota

Andare a capo nel codice senza lasciare una riga vuota **non** crea un nuovo paragrafo, il testo continua sulla stessa riga nel PDF. Un po' come in latex. Per iniziare un paragrafo nuovo, serve la riga vuota.

4.1.2. Andare a capo senza nuovo paragrafo

Se vuoi andare a capo senza creare un nuovo paragrafo (come in una poesia o un indirizzo), usa il backslash `\` seguito da uno spazio:

► Interruzione di riga**CODICE TYPST**

```
Via Roma, 42 \  
00100 Roma \  
Italia
```

RISULTATO

Via Roma, 42
00100 Roma
Italia

In alternativa puoi usare la funzione `#linebreak()`.

4.2. Grassetto, corsivo e sottolineato

4.2.1. Grassetto

Racchiudi il testo tra asterischi `*...*`:

► Grassetto**CODICE TYPST****RISULTATO**

Questa parola è **in grassetto**. Anche più parole insieme funzionano.

Questa parola è ***in grassetto***.
Anche ***più parole insieme***
funzionano.

4.2.2. Corsivo

Racchiudi il testo tra underscore `_`:

► Corsivo

CODICE TYPST

Questa parola è `_in corsivo_`.
Anche `_una frase intera_` può essere in
corsivo.

RISULTATO

Questa parola è *in corsivo*. Anche *una
frase intera* può essere in corsivo.

4.2.3. Combinare grassetto e corsivo

Puoi combinarli annidandoli uno dentro l'altro:

► Grassetto + Corsivo

CODICE TYPST

Questo è `*_grassetto e corsivo_*`
insieme.
Oppure `_testo con parte in
*_grassetto*_`.

RISULTATO

Questo è ***grassetto e corsivo*** insieme.
Oppure *testo con parte in **grassetto***.

4.2.4. Testo cancellato (strikethrough)

Racchiudi il testo dentro `#strike[testo]` (non ho idea a cosa possa servirvi, ma tant'è):

► Testo cancellato

CODICE TYPST

Il prezzo era `#strike[100 euro]`, ora è 50
euro.

RISULTATO

Il prezzo era ~~100 euro~~, ora è 50 euro.

4.2.5. Codice inline

Racchiudi tra backtick (si, l'apice inverso si chiama backtick) ``` `...` ```:

► Codice inline

CODICE TYPST

Usa la funzione ``print()`` per stampare.
Il file si chiama ``documento.typ``.

RISULTATO

Usa la funzione `print()` per stampare.
Il file si chiama `documento.typ`.

4.2.6. Apice e pedice

Per l'apice usa `#super[...]`, per il pedice `#sub[...]`:

► Apice e pedice

CODICE TYPST

L'acqua è H`#sub[2]`O.
La formula è $E = mc$ `#super[2]`.
 x `#sub[1]` + x `#sub[2]` = x `#sub[1+2]`

RISULTATO

L'acqua è H₂O. La formula è $E = mc^2$.
 $x_1 + x_2 = x_{1+2}$

Nota

Per le formule matematiche vere e proprie (con lettere greche, frazioni, integrali...) esistono le modalità matematiche `$...$` che vedremo nel Capitolo 9. L'apice e il pedice con `#super` e `#sub` sono solo per testo normale, usateli con parsimonia.

4.3. Caratteri speciali e come usarli

Alcuni caratteri hanno un significato speciale in Typst. Se vuoi inserirli come testo normale, devi «escaparli» (fare l'escape, trattarli come testo normale) con il backslash `\`:

Carattere	Come scriverlo	Significato speciale
<code>*</code>	<code>*</code>	Grassetto
<code>_</code>	<code>_</code>	Corsivo
<code>~</code>	<code>\~</code>	Spazio inescabibile
<code>`</code>	<code>\`...`</code>	Codice inline
<code>\$</code>	<code>\\$</code>	Matematica
<code>#</code>	<code>\#</code>	Istruzione codice

@	\@	Riferimento
<	\<	Etichetta
\	\\	Backslash letterale

4.4. Virgolette tipografiche

Typst converte automaticamente le virgolette diritte in virgolette tipografiche (curve) appropriate per la lingua del documento:

► Virgolette tipografiche

CODICE TYPST

"Ciao" diventano "virgolette basse".
'Anche' le 'singole' funzionano uguale.

RISULTATO

«Ciao» diventano «virgolette italiane».
“Anche” le “singole” funzionano.

Nota

Le virgolette si adattano alla lingua impostata con `#set text(lang: "it")`. Con la lingua italiana, le doppie virgolette `"..."` diventano «virgolette angolari» (caporali), mentre le singole `'...'` diventano «virgolette curve alte» (le stesse che in inglese si ottengono dalle doppie virgolette), **non** <angolari singole>. Con `lang: "en"` si ottengono le «curly quotes» inglesi.

4.5. Spazi speciali

A volte hai bisogno di tipi di spaziatura particolari:

Tipo	Sintassi	Uso
Spazio normale	<code>lascia lo spazio</code> (barra spazio)	Tra parole
Spazio non divisibile	<code>~</code>	10 kg, Dr. Rossi
Spazio fine	<code>#h(0.2em)</code>	Dopo punti in abbreviazioni
Spazio orizzontale	<code>#h(1cm)</code>	Spaziatura personalizzata
Spazio verticale	<code>#v(1cm)</code>	Spaziatura verticale
Spazio infinito	<code>#h(1fr)</code>	Spinge il testo ai lati

► Spazi speciali

CODICE TYPST	RISULTATO
Dott.~Rossi pesa 80~kg. Sinistra #h(1fr) Destra	Dott. Rossi pesa 80 kg. Sinistra Destra

4.6. Linea orizzontale

Per inserire una linea orizzontale separatrice si usa la funzione `#line()`:

► Linea orizzontale	
CODICE TYPST	RISULTATO
Testo sopra la linea. <code>#line(length: 100%, stroke: 0.5pt)</code> Testo sotto la linea.	Testo sopra la linea. Testo sotto la linea.

Attenzione

A differenza di Markdown, in Typst tre trattini `---` **non** producono una linea orizzontale: producono un trattino lungo (*em dash*, «—»). Due trattini `--` danno invece un trattino medio (*en dash*, «–»). Per una vera riga di separazione usa sempre `#line(length: 100%)`.

5. Titoli e struttura del documento

I titoli danno struttura al documento. In Typst si creano con il simbolo `=` (uguale) all'inizio di una riga.

5.1. I livelli di titolo

Typst supporta sei livelli di titolo, creati con un numero crescente di `=`:

► Livelli di titolo

```
= Capitolo (livello 1)
== Sezione (livello 2)
=== Sottosezione (livello 3)
==== Paragrafo (livello 4)
===== Livello 5
===== Livello 6
```

Attenzione

Il simbolo `=` deve essere all'inizio di una riga e seguito da uno spazio. `=Titolo` (senza spazio) non funziona come titolo, ma viene stampato come testo normale (= ciao, visto?).

5.2. Numerazione automatica

Typst può numerare in modo automatico i titoli. Basta aggiungere nelle impostazioni:

► Numerazione titoli

```
#set heading(numbering: "1.1.1.")
= Primo capitolo
== Prima sezione
=== Prima sottosezione
== Seconda sezione
= Secondo capitolo
```

Esistono vari stili di numerazione:

Stringa di numerazione	Descrizione	Esempio
<code>"1."</code>	Arabi, un livello	1. 2. 3.
<code>"1.1."</code>	Arabi, due livelli	1.1. 1.2. 2.1.
<code>"1.1.1."</code>	Arabi, tre livelli	1.1.1.
<code>"I."</code>	Romani maiuscoli	I. II. III.
<code>"i."</code>	Romani minuscoli	i. ii. iii.
<code>"A."</code>	Lettere maiuscole	A. B. C.
<code>"a."</code>	Lettere minuscole	a. b. c.
<code>"(1)"</code>	Arabi tra parentesi	(1) (2) (3)

5.3. L'indice automatico con `#outline()`

Uno dei poteri di Typst è generare automaticamente l'indice del documento. Metti questa riga dove vuoi che appaia l'indice:

```
#outline()
```

L'indice include tutti i titoli e le loro pagine. Ovviamente è personalizzabile:

```
// Indice fino al livello 3
#outline(depth: 3)
// Indice con titolo personalizzato
#outline(title: "Sommario")
// Indice con rientro automatico
#outline(depth: 3, indent: 2em)
// Indice con solo un certo tipo di elemento
#outline(title: "Lista delle figure", target: figure.where(kind: image))
#outline(title: "Lista delle tabelle", target: figure.where(kind: table))
```

5.4. Titoli fuori dall'indice

A volte vuoi un titolo visivo che non compaia nell'indice (per esempio nella prefazione o nell'appendice):

```
// Titolo non incluso nell'indice e non numerato
#heading(level: 1, numbering: none, outlined: false)[Prefazione]
```

5.5. Etichette e riferimenti incrociati

Puoi assegnare un'etichetta a qualsiasi titolo (o figura, o equazione) e poi **referirtici** da un'altra parte del documento.

5.5.1. Assegnare un'etichetta

Dopo il titolo, scrivi `<nome-etichetta>` tra parentesi angolari:

```
= Capitolo Introduttivo <cap-intro>
== La storia di Typst <storia>
Vedi la sezione sulla storia (@storia) nel capitolo @cap-intro.
```

5.5.2. Fare riferimento

Usa `@nome-etichetta` per riferirti al riferito. Typst genera automaticamente il testo del link con il numero corretto:

- Se punta a un titolo di capitolo: «Capitolo 1»
- Se punta a una figura: «Figura 3»
- Se punta a un'equazione: «Equazione (5)»

► Riferimento a una sezione

```
== Il Teorema di Pitagora <teorema-pitagora>  
Come dimostrato in @teorema-pitagora, il quadrato  
dell'ipotenusa è uguale alla somma dei quadrati  
dei cateti.
```

Nota

Come nella programmazione normale, le etichette dei riferimenti devono essere **univoche** nel documento. Non puoi avere due `<intro>` nello stesso file. Quindi buona norma è usare nomi descrittivi come `<cap-risultati>` invece di `<1>` (e se puoi, commenta! Almeno chi leggerà il documento in futuro e vorrà modificarlo non impazzirà).

5.6. Interruzione di pagina

Per forzare l'inizio di un nuovo capitolo su una nuova pagina:

```
#pagebreak()  
= Nuovo capitolo
```

Con `weak: true`, la pagina si spezza solo se non è già all'inizio di una pagina:

```
#pagebreak(weak: true)
```

6. Liste

Le liste sono fondamentali in quasi ogni documento. Typst ne supporta tre tipi principali.

6.1. Lista puntata

Inizia ogni voce con un trattino `-` seguito da uno spazio:

► Lista puntata

CODICE TYPST	RISULTATO
- Prima voce - Seconda voce - Terza voce	• Prima voce • Seconda voce • Terza voce

6.1.1. Marcatori personalizzati

Il marcatore predefinito è `•`, ma puoi cambiarlo con `set list`:

```
#set list(marker: "→")  
- Prima voce  
- Seconda voce
```

Oppure puoi specificare marcatori diversi per ogni livello:

```
#set list(marker: ([•], [-], [○]))
```

6.2. Lista numerata

Inizia ogni voce con `+` seguito da uno spazio:

► Lista numerata	
CODICE TYPST	RISULTATO
+ Primo punto + Secondo punto + Terzo punto	1. Primo punto 2. Secondo punto 3. Terzo punto

6.2.1. Stile di numerazione

```
// Numerazione con lettere  
#set enum(numbering: "a")  
+ Prima voce // a)  
+ Seconda voce // b)  
// Numerazione con romani  
#set enum(numbering: "I.")
```

6.2.2. Iniziare da un numero specifico

```
#enum(start: 5,  
  [Quinta voce],  
  [Sesta voce],  
  [Settima voce],  
)
```

6.3. Liste annidate

Per creare una sottolista, indenta le voci con due spazi (o un tab):

► Lista annidata

CODICE TYPST

- Frutta
 - Mele
 - Pere
 - Banane
- Verdura
 - Carote
 - Spinaci
- Cereali

RISULTATO

- Frutta
 - Mele
 - Pere
 - Banane
- Verdura
 - Carote
 - Spinaci
- Cereali

Puoi mescolare liste puntate e numerate:

► Mescolare liste

CODICE TYPST

- + Prepara gli ingredienti:
 - Farina 500g
 - Acqua 300ml
 - Sale 10g
- + Impasta
- + Lascia riposare:
 - Almeno 2 ore
 - In luogo fresco
- + Cuoci

RISULTATO

1. Prepara gli ingredienti:
 - Farina 500g
 - Acqua 300ml
 - Sale 10g
2. Impasta
3. Lascia riposare:
 - Almeno 2 ore
 - In luogo fresco
4. Cuoci

6.4. Lista di definizioni (termine: definizione)

Per creare un glossario o elenco di termini con definizioni, usa `/ termine: definizione`:

► Lista di definizioni

CODICE TYPST

`/ Markup`: Un linguaggio che usa marcatori speciali nel testo per descriverne la struttura e la formattazione.

`/ Compilatore`: Programma che trasforma il codice sorgente in un file di output (nel caso di Typst, in un PDF).

`/ PDF`: Portable Document Format. Formato di file universalmente leggibile.

RISULTATO

Markup Un linguaggio che usa marcatori speciali nel testo per descriverne la struttura e la formattazione.

Compilatore Programma che trasforma il codice sorgente in un file di output (nel caso di Typst, in un PDF).

PDF Portable Document Format. Formato di file universalmente leggibile.

6.5. Voci su più righe

Le voci di una lista possono contenere più paragrafi. Basta far rientrare il testo successivo allo stesso livello:

► Voci multi-paragrafo

CODICE TYPST

`- *Prima voce importante*`
Questo è il paragrafo di spiegazione per la prima voce. Può essere lungo quanto vuoi.
Puoi anche avere più paragrafi per la stessa voce.

`- *Seconda voce*`
Un'altra spiegazione.

RISULTATO

- **Prima voce importante** Questo è il paragrafo di spiegazione per la prima voce. Può essere lungo quanto vuoi. Puoi anche avere più paragrafi per la stessa voce.
- **Seconda voce** Un'altra spiegazione.

6.6. Spaziatura tra le voci

Di default le voci di lista sono compatte. Puoi aumentare la spaziatura tra le voci:

```
#set list(spacing: 1.5em)
```

- Prima voce
- Seconda voce (ora c'è più spazio tra le voci)

7. Link e riferimenti

7.1. Link a pagine web

7.1.1. Sintassi base

```
#link("https://typst.app")[Visita Typst]
```

Il primo argomento (tra parentesi tonde) è l'URL, il secondo (tra parentesi quadre) è il testo da mostrare:

► Link a URL

CODICE TYPST

```
Visita #link("https://typst.app")[il sito  
ufficiale di Typst].  
Oppure: documentazione su  
#link("https://typst.app/docs")  
[typst.app/docs].
```

RISULTATO

Visita [il sito ufficiale di Typst](https://typst.app). Oppure:
documentazione su typst.app/docs.

7.1.2. URL automatici

Se scrivi un URL direttamente nel testo, Typst lo riconosce e lo rende cliccabile automaticamente:

Vai su <https://typst.app> per iniziare.

7.1.3. Link senza testo visibile (solo URL)

```
#link("https://typst.app")  
// Mostra l'URL stesso come testo
```

7.1.4. Link a indirizzi email

```
Scrivimi a #link("mailto:info@esempio.it")[info@esempio.it]
```

7.2. Note a piè di pagina

Le note a piè di pagina si inseriscono con `#footnote[...]`:

► Note a piè di pagina

CODICE TYPST

```
Il termine "markup"#footnote[  
  Dal termine inglese "mark up", che  
  significa  
  "annotare" o "contrassegnare".  
] risale agli anni '60.
```

RISULTATO

Il termine «markup»⁴ risale agli anni
"60.

Le note vengono automaticamente numerate e appaiono in fondo alla pagina corrente.

7.3. Riepilogo link e riferimenti

Sintassi	Effetto
<code>#link("url")[testo]</code>	Link esterno con testo
<code>https://...</code>	URL automatico cliccabile
<code><etich></code>	Assegna etichetta
<code>@etich</code>	Riferimento all'etichetta
<code>#footnote[testo]</code>	Nota a piè di pagina

⁴Dal termine inglese «mark up», che significa «annotare» o «contrassegnare».

8. Immagini e figure

8.1. Inserire un'immagine

La funzione per inserire immagini è `#image("percorso")`. Il percorso è relativo alla posizione del file `.typ`:

```
// Immagine nella stessa cartella del file .typ
#image("foto.jpg")
// Immagine in una sottocartella
#image("immagini/grafico.png")
// Immagine con larghezza fissa
#image("logo.png", width: 5cm)
// Immagine con larghezza percentuale
#image("schema.png", width: 80%)
// Immagine con altezza fissa
#image("foto.jpg", height: 4cm)
```

8.1.1. Formati supportati

Typst supporta i formati immagine più comuni:

- PNG (`.png`)
- JPEG/JPG (`.jpg` , `.jpeg`)
- SVG (`.svg`) — vettoriale, ideale per loghi e schemi
- GIF (`.gif`) — solo il primo frame

8.1.2. Parametro `fit`

Quando specifichi sia `width` che `height`, puoi controllare come l'immagine viene ridimensionata (così da non creare caos al cambio di dispositivo) con il parametro `fit`:

```
// "cover": riempie l'area (può tagliare)
#image("foto.jpg", width: 10cm, height: 6cm, fit: "cover")
// "contain": l'immagine intera (con spazio vuoto laterale)
#image("foto.jpg", width: 10cm, height: 6cm, fit: "contain")
// "stretch": deforma l'immagine per riempire
#image("foto.jpg", width: 10cm, height: 6cm, fit: "stretch")
```

8.2. La funzione `#figure()`: figure con didascalia

Per le figure professionali (con didascalia, numerazione automatica, e inclusione nell'indice delle figure) usa `#figure()`:

```
#figure(  
  image("grafico.png", width: 80%),  
  caption: [Andamento delle temperature nel 2024.],  
) <fig-temperature>
```

8.2.1. Componenti di `#figure()`

La funzione `figure` accetta:

- Il **contenuto** come primo argomento (l'immagine, la tabella, ecc.)
- `caption`: la didascalia (tra parentesi quadre)
- `label`: puoi anche mettere l'etichetta `<nome>` dopo `)`

8.2.2. Personalizzare il supplemento

Di default le figure mostrano «Figura 1», «Figura 2», ecc. Puoi personalizzare:

```
#set figure(supplement: "Fig.")  
// Ora mostra "Fig. 1", "Fig. 2"  
// In italiano, Typst potrebbe già usare "Figura"  
// se hai impostato lang: "it"
```

8.2.3. Posizionamento delle figure

Per impostazione predefinita, le figure vengono posizionate «in linea» con il testo. Puoi forzare il posizionamento:

```
#figure(  
  image("schema.png"),  
  caption: [Schema del sistema.],  
  placement: top, // Metti in cima alla pagina  
)  
// Altre opzioni: bottom, none (inline)
```

8.3. Allineamento

Puoi centrare o allineare il contenuto con `#align()`:

```
// Centrato  
#align(center)[  
  #image("logo.png", width: 4cm)
```

```
]
// A destra
#align(right)[
  #image("firma.png", width: 3cm)
]
```

8.4. Immagini affiancate

Per mettere due immagini affiancate, usa `#grid()`:

```
#grid(
  columns: (1fr, 1fr),
  gutter: 1em,
  figure(image("prima.png"), caption: [Prima immagine.]),
  figure(image("seconda.png"), caption: [Seconda immagine.]),
)
```

8.5. Formati e ottimizzazione delle immagini

8.5.1. Immagini SVG per grafica vettoriale

Il formato SVG è preferibile per loghi, schemi, diagrammi e qualsiasi grafica che deve restare nitida a qualsiasi scala. Typst include il renderer SVG nativamente, nessun pacchetto extra è necessario:

```
// SVG: nitido a qualsiasi risoluzione
#image("schema.svg", width: 100%)

// Puoi disegnare SVG inline nel documento
#image(bytes("<svg xmlns='http://www.w3.org/2000/svg'
width='100' height='100'>
  <circle cx='50' cy='50' r='40'
  stroke='#1a3a5c' stroke-width='3' fill='#aed6f1'/>
</svg>"))
```

8.5.2. Immagini: solo file locali (niente URL)

Attenzione

A differenza di altri strumenti, Typst **non** scarica immagini da internet: `#image()` accetta solo percorsi a file locali (o dati in memoria). Passare un URL produce l'errore `file not found`, perché Typst lo interpreta come un percorso sul disco.

Se vuoi usare un'immagine presa dal web, scaricala prima e poi referenziala localmente:

```
// Sbagliato: Typst NON scarica da rete → errore "file not found"
// #image("https://esempio.com/logo.svg")

// Corretto: scarica il file nella cartella del progetto e usa il path
#image("immagini/logo.svg", width: 4cm)
```

8.5.3. Testo alternativo (accessibilità)

Per la conformità PDF/UA-1 è buona pratica aggiungere una descrizione testuale all'immagine:

```
#image("grafico-dati.png",
  width: 80%,
  alt: "Grafico a barre che mostra le vendite mensili del 2024",
)
```

8.5.4. Clip e zoom: mostrare una porzione di immagine

```
// Mostra solo la parte centrale dell'immagine
#box(
  clip: true,
  width: 5cm,
  height: 4cm,
)[
  #move(dx: -2cm, dy: -1cm)[
    #image("foto-larga.jpg", width: 9cm)
  ]
]
```

8.5.5. Immagini affiancate con didascalie individuali

```
#grid(
  columns: (1fr, 1fr),
  gutter: 1.5em,
  figure(
    image("prima.png"),
    caption: [Prima condizione sperimentale.],
  ) <fig-prima>,
  figure(
    image("seconda.png"),
    caption: [Seconda condizione sperimentale.],
  ) <fig-seconda>,
)
```

I risultati in @fig-prima e @fig-seconda mostrano...

8.5.6. Rotazione e trasformazione

```
// Ruota l'immagine
#rotate(90deg, image("schema.png", width: 6cm))

// Scala (specchio orizzontale)
#scale(x: -100%, image("logo.png", width: 3cm))

// Scala con percentuale
#scale(80%, image("foto.jpg", width: 10cm))
```

8.5.7. Immagine come sfondo di pagina

```
#set page(
  background: place(
    center + horizon,
    image("sfondo.png", width: 100%, height: 100%, fit: "cover"),
  ),
)
```

8.6. Consigli pratici

Consiglio

Per i documenti scientifici e accademici, **usa sempre** `#figure()` invece di `#image()` direttamente. Questo ti permette di:

- Avere numerazione automatica delle figure
- Fare riferimento con `@nome-figura` nel testo
- Generare l'indice delle figure con `#outline(target: figure)`

Consiglio

Usa immagini **SVG** quando possibile per loghi, schemi e grafici: restano nitidi a qualsiasi dimensione nel PDF. Per fotografie usa JPEG o PNG (obbligatorio se vuoi il background invisibile).

Nota

Nell'editor online di `typst.app` puoi caricare le immagini direttamente trascinandole sull'editor o usando il pulsante di upload. I file caricati sono accessibili dal tuo progetto. La gestione ordinata del progetto per cartelle è importante anche per questo.

9. Tabelle

Le tabelle in Typst sono potenti e flessibili. La funzione principale è `#table()`.

9.1. La prima tabella

```
#table(  
  columns: 3, // numero di colonne  
  [Nome], [Età], [Città],  
  [Mario], [25], [Roma],  
  [Lucia], [30], [Milano],  
  [Carlo], [22], [Napoli],  
)
```

Ogni elemento tra `[...]` è una cella. Le celle vengono riempite riga per riga, da sinistra a destra.

► Tabella base

CODICE TYPST

```
#table(  
  columns: 3,  
  [Nome], [Età], [Città],  
  [Mario], [25], [Roma],  
  [Lucia], [30], [Milano],  
  [Carlo], [22], [Napoli],  
)
```

RISULTATO

Nome	Età	Città
Mario	25	Roma
Lucia	30	Milano
Carlo	22	Napoli

9.2. Definire le colonne

Il parametro `columns` accetta:

- Un numero intero (colonne di larghezza uguale)
- Una lista di misure specifiche

```
// 3 colonne uguali  
columns: 3  
// Larghezze specifiche  
columns: (3cm, 2cm, 4cm)  
// Proporzioni: 1fr = "il resto dello spazio"
```

```
columns: (1fr, 2fr, 1fr)
// Prima e terza colonna uguali, seconda larga il doppio
// Mix di fisso e proporzionale
columns: (auto, 1fr, 2cm)
// Prima colonna: si adatta al contenuto
// Seconda: prende lo spazio rimanente
// Terza: fissa 2cm
```

9.3. Intestazioni della tabella

Per creare intestazioni stilizzate, usa `table.header()`:

► Tabella con intestazione colorata

CODICE TYPST

```
#table(
  columns: (1fr, auto, auto),
  stroke: 0.6pt + luma(180),
  fill: (col, row) => if row == 0
  { rgb("#1a3a5c") }
  else if calc.odd(row) { white }
  else { rgb("#f4f6f7") },
  table.header(
    [*Prodotto*], [*Prezzo*],
    [*Disponibile*],
  ),
  [Laptop], [999 €], [Sì],
  [Mouse], [25 €], [Sì],
  [Monitor], [350 €], [No],
  [Tastiera], [45 €], [Sì],
)
```

RISULTATO

Prodotto	Prezzo	Disponibile
Laptop	999 €	Sì
Mouse	25 €	Sì
Monitor	350 €	No
Tastiera	45 €	Sì

9.4. Allineamento del contenuto

```
#table(
  columns: 3,
  align: (left, center, right),
  [Testo a sinistra], [Testo centrato], [Testo a destra],
  [Mario], [25], [Roma],
)
```

Oppure un allineamento globale:

```
#table(
  columns: 3,
  align: center, // Tutte le celle centrate
  ...
)
```

Allineamento verticale:

```
align: (left + top, center + horizon, right + bottom)
```

9.5. Colori alternati (zebra stripes)

Il parametro `fill` accetta una funzione che riceve riga e colonna e restituisce un colore:

```
#table(
  columns: 3,
  fill: (col, row) => if calc.odd(row) { luma(240) } else { white },
  [A], [B], [C],
  [1], [2], [3],
  [4], [5], [6],
)
```

9.6. Bordi (stroke)

```
// Bordo uniforme
stroke: 0.5pt + black
// Nessun bordo
stroke: none
// Bordo solo orizzontale
stroke: (top: 0.5pt + black, bottom: 0.5pt + black)
// Bordo personalizzato per cella
// (nella funzione fill puoi tornare stroke)
```

9.7. Celle che si estendono su più colonne/righe

```
#table(
  columns: 3,
  // Questa cella occupa 2 colonne
  table.cell(colspan: 2)[Colonne 1 e 2 unite],
  [Colonna 3],
  [Normale], [Normale], [Normale],
)
```

```
// Questa cella occupa 2 righe
table.cell(rowspan: 2)[2 righe],
[Riga 1], [Riga 1],
[Riga 2], [Riga 2],
)
```

9.8. Tabelle come figure

Per avere numerazione automatica e poter fare riferimento a una tabella, wrappala in `#figure()`:

```
#figure(
  table(
    columns: 3,
    [A], [B], [C],
    [1], [2], [3],
  ),
  caption: [Dati sperimentali.],
) <tab-dati>
Come mostrato in @tab-dati...
```

Nota

`#figure(table(...), ...)` identifica automaticamente il contenuto come tabella (non come immagine), quindi nell'indice delle tabelle verrà elencato separatamente dalle figure.

10. Matematica

Uno dei punti di forza di Typst è la sua sintassi matematica: più pulita di LaTeX, facile da imparare, e con risultati tipograficamente eccellenti.

10.1. Modalità matematica

Come avevamo accennato in precedenza, ci sono (essenzialmente) due modi per scrivere matematica in Typst:

10.1.1. Matematica inline

La matematica **inline** appare nel corpo del testo, circondata da `$...$` (dollaro singolo, senza spazi):

► Matematica inline

CODICE TYPST

La formula di Eulero è $e^{i\pi} + 1 = 0$,
 considerata la più bella della
 matematica.
 Il teorema di Pitagora: $a^2 + b^2 = c^2$.

RISULTATO

La formula di Eulero è $e^{i\pi} + 1 = 0$,
 considerata la più bella della matema-
 tica. Il teorema di Pitagora: $a^2 + b^2 = c^2$.

10.1.2. Matematica a blocco (display)

La matematica **a blocco** appare su una riga propria, centrata e con simboli più grandi. Si ottiene con `$...$` su una riga separata, oppure aggiungendo uno spazio dopo il `$` iniziale:

► Matematica a blocco

CODICE TYPST

L'equazione di Einstein:
 $E = mc^2$
 Oppure l'identità di Eulero in grande:
 $e^{i\pi} + 1 = 0$

RISULTATO

L'equazione di Einstein:

$$E = mc^2$$

 Oppure l'identità di Eulero in grande:

$$e^{i\pi} + 1 = 0$$

Nota

La differenza tra inline e display è lo spazio dopo `$`:

- `$formula$` = inline (nel flusso del testo)
- `$ formula $` = display (su riga propria, centrata)

Consiglio

È sempre preferibile, per la chiarezza del testo, quando si introduce una nuova equazione/funzione matematica, utilizzare il display (blocco). Rimane tutto molto più ordinato. Il mio consiglio è: utilizzate sempre il display, occupa più spazio ma è più leggibile. L'inline funziona bene se avere poca matematica o state solamente facendo riferimento ad altro (che era in un display).

10.2. Operatori di base

Operazione	Sintassi Typst	Risultato
Addizione	<code>a + b</code>	$a + b$
Sottrazione	<code>a - b</code>	$a - b$
Moltiplicazione	<code>a dot b</code>	$a \cdot b$
Prodotto vettoriale	<code>a times b</code>	$a \times b$
Divisione	<code>a / b</code>	$\frac{a}{b}$
Uguale	<code>a = b</code>	$a = b$
Diverso	<code>a != b</code>	$a \neq b$
Minore	<code>a < b</code>	$a < b$
Maggiore	<code>a > b</code>	$a > b$
Minore uguale	<code>a <= b</code>	$a \leq b$
Maggiore uguale	<code>a >= b</code>	$a \geq b$
Approssimato	<code>a approx b</code>	$a \approx b$
Proporzionale	<code>a prop b</code>	$a \propto b$

10.3. Potenze e pedici

// Potenza semplice
`$x^2$` // x^2
`$x^{(n+1)}$` // x elevato a $(n+1)$ — serve la parentesi!
// Pedice semplice
`$x_i$` // x_i
`$x_{(i,j)}$` // con pedice composto
// Insieme
`$x_i^2$` // x_i^2
`$x_{(i+1)}^{(2n)}$` // con espressioni composte

Potenze e pedici

CODICE TYPST
`$x^2 + y^2 = r^2$` è la circonferenza.
`$a_n = a_{(n-1)} + a_{(n-2)}$` è Fibonacci.
La serie `$sum_{(n=0)}^{(+infinity)} x^n$` = $\frac{1}{1-x}$
converge per `$|x| < 1$`.

RISULTATO
 $x^2 + y^2 = r^2$ è la circonferenza. $a_n = a_{n-1} + a_{n-2}$ è Fibonacci. La serie $\sum_{n=0}^{+\infty} x^n = \frac{1}{1-x}$ converge per $|x| < 1$.

10.4. Frazioni

`$a/b$` // Frazione a/b (forma compatta in inline)
`$frac(a, b)$` // Frazione esplicita (sempre con sbarra)
`$(a+b)/(c+d)$` // Frazione con espressioni

► Frazioni e integrali

CODICE TYPST

La derivata: `$frac(d f, d x) = \lim_{(h \rightarrow 0)} frac(f(x+h)-f(x), h)$`
 L'integrale: `$integral_a^b f(x) d x = F(b) - F(a)$`

RISULTATO

La derivata: $\frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h)-f(x)}{h}$
 L'integrale:

$$\int_a^b f(x) dx = F(b) - F(a)$$

10.5. Radici

`$sqrt(x)$` // Radice quadrata
`$sqrt(x + y)$` // Radice di un'espressione
`$root(3, x)$` // Radice cubica
`$root(n, x^n)$` // Radice n-esima

► Formula quadratica

CODICE TYPST

La soluzione è `$x = frac(-b plus.minus sqrt(b^2 - 4a c), 2a)$`.

RISULTATO

La soluzione è $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$.

10.6. Simboli speciali comuni

10.6.1. Lettere greche

Simbolo	Codice	Simbolo	Codice	Simbolo	Codice
α	<code>alpha</code>	β	<code>beta</code>	γ	<code>gamma</code>
δ	<code>delta</code>	ε	<code>epsilon</code>	ζ	<code>zeta</code>

η	eta	θ	theta	λ	lambda
μ	mu	ν	nu	ξ	xi
π	pi	ρ	rho	σ	sigma
τ	tau	φ	phi	ψ	psi
ω	omega	Δ	Delta	Σ	Sigma
Ω	Omega	Λ	Lambda	Π	Pi

10.6.2. Altri simboli frequenti

Simbolo	Codice	Simbolo	Codice
∞	infinity	∂	partial
∇	nabla	$\forall$	forall
$\exists$	exists	$\in$	in
$\subset$	subset	$\cup$	union
$\cap$	inter	$\rightarrow$	arrow.r
$\leftrightarrow$	arrow.l.r	$\Rightarrow$	arrow.r.double
$\pm$	plus.minus	$\mp$	minus.plus

10.7. Sommatorie, produttorie, limiti

```
// Sommatoria
$sum_(i=0)^n a_i$
// Produttoria
$product_(k=1)^n k = n!$
// Limite
$lim_(x -> 0) frac(sin x, x) = 1$
// In modalità display (più grande)
$sum_(i=0)^n i^2 = frac(n(n+1)(2n+1), 6)$
```

► Sommatoria e limite

CODICE TYPST

```
$sum_(i=1)^n i = frac(n(n+1), 2)$
$lim_(n->infinity) (1 + 1/n)^n = e$
```

RISULTATO

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n = e$$

10.8. Vettori e matrici

```
// Vettore colonna
$vec(a, b, c)$
// Matrice
$mat(a, b; c, d)$ // Il punto e virgola separa le righe
// Matrice con bordi
$mat(delim: "[", a, b; c, d)$
$mat(delim: "|", a, b; c, d)$ // Determinante
```

► Matrici e vettori

CODICE TYPST

La matrice di rotazione è:
 $R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$
 Un sistema lineare:
 $\begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} e \\ f \end{pmatrix}$

RISULTATO

La matrice di rotazione è:

$$R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

Un sistema lineare:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} e \\ f \end{pmatrix}$$

10.9. Testo all'interno delle formule

Per inserire testo normale dentro una formula usa le virgolette:

```
$f(x) = x^2 quad "per" x > 0$
```

► Funzione a tratti

CODICE TYPST

```
$f(x) = cases(x^2 &"se" x >= 0, -x^2  
&"se" x < 0)$
```

RISULTATO

$$f(x) = \begin{cases} x^2 & \text{se } x \geq 0 \\ -x^2 & \text{se } x < 0 \end{cases}$$

10.10. Formule numerate

Per le formule numerate in un documento scientifico:

```
#set math.equation(numbering: "(1)")  
$ E = m c^2 $ <eq-einstein>  
Come dimostrato in @eq-einstein...
```

10.11. Spazi in matematica

In modalità matematica gli spazi normali vengono ignorati. Per aggiungere spaziatura usa:

```
$a quad b$ // spazio grande  
$a wide b$ // spazio molto grande  
$a thin b$ // spazio sottile  
$a med b$ // spazio medio  
$a space b$ // spazio normale
```

11. Blocchi di codice

Typst gestisce ottimamente la visualizzazione di codice sorgente con evidenziazione della sintassi (**syntax highlighting**) per decine di linguaggi.

11.1. Codice inline

Usa un backtick singolo per codice inline nel testo:

```
Usa la funzione print() per visualizzare output.  
La variabile x ha valore 42.
```

11.2. Blocco di codice

Usa tre backtick per un blocco di codice, con il nome del linguaggio dopo i backtick iniziali:

```
```python  
def fattoriale(n):
 if n <= 1:
 return 1
 return n * fattoriale(n - 1)
print(fattoriale(5)) # Output: 120
```
```

► Codice Python con highlighting

```
```python
def saluta(nome):
 return f"Ciao, {nome}!"

print(saluta("Typst"))
```
```

11.3. Linguaggi supportati

Typst supporta l'evidenziazione per moltissimi linguaggi:

| Linguaggio | Identificatore | Linguaggio | Identificatore |
|------------|-----------------------|------------|---|
| Python | <code>python</code> | JavaScript | <code>javascript</code> o <code>js</code> |
| Java | <code>java</code> | C/C++ | <code>c</code> , <code>cpp</code> |
| Rust | <code>rust</code> | Go | <code>go</code> |
| HTML | <code>html</code> | CSS | <code>css</code> |
| SQL | <code>sql</code> | Bash | <code>bash</code> , <code>sh</code> |
| LaTeX | <code>latex</code> | Typst | <code>typ</code> |
| JSON | <code>json</code> | YAML | <code>yaml</code> |
| Markdown | <code>markdown</code> | R | <code>r</code> |
| MATLAB | <code>matlab</code> | Julia | <code>julia</code> |

11.4. Numeri di riga

Con il pacchetto `codly` (che vedremo nel Capitolo 25) puoi aggiungere numeri di riga e molte altre opzioni. Per ora, la visualizzazione base di Typst non include numeri di riga.

11.5. Stile personalizzato del codice

Puoi personalizzare l'aspetto di tutto il codice usando `#set raw`:

```
// Cambia il font del codice
#set raw(theme: "halcyon") // tema scuro
// Oppure via show rule (come fatto in questo manuale)
#show raw.where(block: true): it => {
  block(
    fill: luma(245),
```

```
inset: 1em,  
radius: 5pt,  
)[#it]  
}
```

11.6. Numero di riga e evidenziazione manuale

Puoi simulare i numeri di riga con un layout a griglia:

```
#let codice-con-numeri(src) = {  
  let lines = src.split("\n")  
  let celle = ()  
  for (i, line) in lines.enumerate() {  
    celle.push(text(fill: luma(150), str(i+1)))  
    celle.push(raw(line))  
  }  
  grid(columns: (auto, 1fr), gutter: 0.5em, ..celle)  
}
```

12. Introduzione al codice: il simbolo `#`

Fino a qui hai usato Typst come un linguaggio di markup: scrivi testo, aggiungi `*grassetto*`, `= Titoli`, e via dicendo. Ma Typst ha un secondo volto: è anche un linguaggio di **programmazione**. In questo capitolo iniziamo a esplorarlo.

12.1. Due modalità: Markup e Codice

In Typst esistono due modalità di funzionamento:

| | Modalità Markup | Modalità Codice |
|----------------------|--|---|
| Come si entra | È la default | Con il simbolo <code>#</code> |
| Cosa scrivi | Testo normale con marcature | Istruzioni di programmazione |
| Esempi | <code>*grassetto*</code> , <code>= Titolo</code> | <code>#let x = 5</code> , <code>#if cond [...]</code> |
| Scopo | Contenuto del documento | Logica, variabili, funzioni |

Il simbolo `#` è il **cancelletto** (hash). Quando lo scrivi nel testo, stai dicendo a Typst: «quello che segue non è testo, è un'istruzione di codice».

12.2. Il simbolo `#` in azione

Ecco come funziona il passaggio tra le due modalità:

```
Questo è testo normale.  
#let nome = "Fabiana" // Definisco una variabile  
Ciao, #nome! Come stai? Sei molto bella! // Uso la variabile nel testo  
Calcolo: #(2 + 3) fa cinque. // Calcolo inline  
#if true [Questa frase appare.] // Condizione inline
```

Ogni riga che inizia con `#` (o che ha `#` nel mezzo del testo) contiene codice Typst.

12.3. Espressioni inline con `#`

Puoi inserire il risultato di qualsiasi espressione nel testo usando `#{espressione}` o `#variabile`:

► Espressioni inline

CODICE TYPST

```
Il risultato di 7 × 6 è #(7 * 6).\nL'anno corrente è\n#datetime.today().year.\nIl testo in maiuscolo: #("ciao\nmondo".split(" ").map(s =>\n  upper(s)).join(" ")).
```

RISULTATO

Il risultato di 7 × 6 è 42.
2026.
Il testo in maiuscolo: CIAO MONDO.

12.4. Blocchi di codice con `{ }`

Le parentesi graffe `{ }` contengono un **blocco di codice**: più istruzioni eseguite in sequenza. Il risultato è il valore dell'ultima espressione.

```
#{  
  let a = 10  
  let b = 20  
  a + b // Restituisce 30  
}
```

Nell'output del documento appare il valore restituito:

► Blocco di codice

CODICE TYPST

```
Il massimo tra 7 e 3 è #{
  let a = 7
  let b = 3
  if a > b { a } else { b }
}.
```

RISULTATO

Il massimo tra 7 e 3 è 7.

12.5. Blocchi di contenuto con `[]`

Le parentesi quadre `[]` contengono un **blocco di contenuto**: testo e markup che viene trattato come contenuto del documento. Puoi pensarle come «mini-documenti»:

```
// Contenuto come argomento di una funzione
#block(fill: yellow)[
  Questo testo è dentro un blocco giallo.
  Posso usare *grassetto* e _corsivo_ normalmente.
]

// Contenuto assegnato a una variabile
#let messaggio = [Ciao, *mondo*]
#messaggio // Stampa il contenuto
```

► Blocchi di contenuto

CODICE TYPST

```
#let avviso = [
  *Attenzione:* questo è un avviso
  importante.
]
#block(fill: rgb("#fff3cd"), inset: 0.8em,
radius: 5pt)[#avviso]
```

RISULTATO

Attenzione: questo è un avviso importante.

12.6. La differenza fondamentale: `{ }` vs `[]`

Questa distinzione è **cruciale** in Typst e vale la pena ribadirla con una tabella:

| | Graffe <code>{ }</code> | Quadre <code>[]</code> |
|----------------------|----------------------------|--------------------------|
| Contengono | Codice (istruzioni) | Contenuto (testo/markup) |
| Dentro si usa | Sintassi di programmazione | Sintassi di markup |

| | | |
|-----------------------------|--------------------------------|---------------------|
| *bold* funziona? | No | Sì |
| #let x = 5 funziona? | Sì | Con il # |
| Restituiscono | Valore dell'ultima espressione | Blocco di contenuto |

12.7. Il principio di base: **#** per tutto il codice

Ogni volta che vuoi fare qualcosa di «programmatico» nel mezzo del markup, usi **#**:

```
// Chiamare una funzione:
#upper("ciao") // "CIAO"
// Definire una variabile:
#let pi = 3.14159
// Usare una variabile:
Pi greco vale circa #pi.
// Condizione:
#if 2 > 1 [Vero!] else [Falso!]
// Ciclo:
#for i in (1, 2, 3) [Numero #i.]
// Funzione integrata:
#text(fill: red)[Testo rosso]
// Impostazione:
#set text(size: 14pt)
```

Nota

Se sei nel mezzo di un blocco di codice (**#{ ... }**), non hai bisogno del **#** davanti a ogni cosa, sei già in modalità codice. Il **#** serve solo quando sei in modalità markup e vuoi «uscire» momentaneamente in codice.

12.8. Il contesto di esecuzione

Typst esegue il codice nell'ordine in cui appare nel documento. Le variabili definite con **#let** rimangono disponibili per tutto il documento (o per lo scope in cui sono definite):

► Variabili nel documento

CODICE TYPST

```
#let titolo = "Il Mio Libro: Frammenti
brevi cronotropi"
#let autore = "Stefano Coelati Rama"
#let anno = 2026
```

RISULTATO

Il Mio Libro: Frammenti brevi cronotropi Scritto da **Stefano Coelati Rama** nel 2026.

```
= #titolo  
Scritto da *#autore* nel #anno.
```

13. Tipi di dato

Come in programmazione normale, anche in Typst ogni valore ha un **tipo**, una categoria che determina cosa puoi farci. Conoscere i tipi è fondamentale per capire perché certi codici funzionano e altri no.

13.1. Panoramica dei tipi

Typst ha questi tipi principali:

| Tipo | Esempio | Descrizione |
|----------|----------------|--------------------------|
| string | "ciao" | Testo tra virgolette |
| int | 42 , -7 | Numero intero |
| float | 3.14 , -0.5 | Numero decimale |
| bool | true , false | Vero o falso |
| length | 2cm , 12pt | Misura di lunghezza |
| ratio | 50% | Percentuale |
| color | rgb("#ff0000") | Colore |
| array | (1, 2, 3) | Lista ordinata di valori |
| dict | (a: 1, b: 2) | Dizionario chiave-valore |
| content | [*Ciao*] | Contenuto del documento |
| none | none | Assenza di valore |
| auto | auto | Valore automatico |
| function | x => x + 1 | Una funzione |

13.2. Stringhe

Una stringa è una sequenza di caratteri racchiusa tra **doppie virgolette** `"..."`:

```
#let nome = "Mario"  
#let saluto = "Ciao, mondo!"  
#let vuota = "" // Stringa vuota
```

```
// Non esiste la singola virgoletta in Typst per le stringhe:  
// 'sbagliato' ← ERRORE!  
// "corretto" ← OK
```

13.2.1. Operazioni sulle stringhe

```
// Concatenazione  
#let nome-completo = "Mario" + " " + "Rossi"  
// Lunghezza  
#"ciao".len() // 4  
// Maiuscolo/minuscolo  
#upper("ciao") // "CIAO"  
#lower("CIAO") // "ciao"  
// Contenuto  
#"ciao mondo".contains("mondo") // true  
// Sostituzione  
#"ciao mondo".replace("mondo", "Typst") // "ciao Typst"  
// Dividi  
#"a,b,c".split(",") // ("a", "b", "c")  
// Elimina spazi  
#" ciao ".trim() // "ciao"
```

13.2.2. Interpolazione di stringhe

Per inserire valori dentro una stringa, è più comodo usare la concatenazione o direttamente il markup:

```
#let nome = "Mario"  
// Nel markup:  
Ciao, #nome!  
// Nella stringa (con concatenazione):  
#let msg = "Ciao, " + nome + "!"
```

13.3. Numeri interi (int)

Un intero è un numero senza parte decimale:

```
#let n = 42  
#let negativo = -7  
#let zero = 0  
// Operazioni:  
#(10 + 3) // 13  
#(10 - 3) // 7  
#(10 * 3) // 30  
#(10 / 3) // 3.333... (diventa float!)  
#(10 // 3) // 3 (divisione intera)
```

```
#(10 mod 3) // 1 (resto della divisione)
#calc.pow(2, 8) // 256 (potenza)
#calc.abs(-5) // 5 (valore assoluto)
```

13.4. Numeri decimali (float)

I float hanno la virgola (o il punto, dipende dalla notazione):

```
#let pi = 3.14159
#let gravita = -9.81
// Costante predefinita:
#calc.pi // 3.141592...
#calc.e // 2.718281...
// Funzioni matematiche:
#calc.sqrt(16.0) // 4.0
#calc.sin(calc.pi / 6) // 0.5
#calc.cos(0.0) // 1.0
#calc.log(100.0) // 2.0 (log base 10)
#calc.ln(calc.e) // 1.0 (log naturale)
#calc.floor(3.7) // 3 (arrotonda giù)
#calc.ceil(3.2) // 4 (arrotonda su)
#calc.round(3.5) // 4 (arrotonda)
```

13.5. Booleani (bool)

Un booleano è semplicemente `true` o `false`:

```
#let sono-logged-in = true
#let ha-errori = false
// Operazioni logiche:
#(true and false) // false
#(true or false) // true
#(not true) // false
// Confronti (restituiscono bool):
#(5 > 3) // true
#(5 == 5) // true (DOPPIO uguale per confronto!)
#(5 != 3) // true
#("a" == "a") // true
```

Attenzione

In Typst (come nella maggior parte dei linguaggi), per **confrontare** due valori si usa il **doppio uguale** `==`, mentre per **assegnare** un valore a una variabile si usa il **singolo uguale** `=`. Confondere i due è un errore comune!

```
// Assegnazione (in let), fai sì che dentro la variabile x ci sia 5:  
#let x = 5  
// Confronto (nelle condizioni), è x uguale a 5? (restituisce il booleano sì/no):  
#if x == 5 [Sì!]
```

13.6. Lunghezze e misure

Le lunghezze sono fondamentali per il layout. Typst supporta varie unità:

| Unità | Esempio | Descrizione |
|-------|---------|--|
| pt | 12pt | Punto tipografico (1/72 pollice) |
| cm | 2.5cm | Centimetro |
| mm | 10mm | Millimetro |
| in | 1in | Pollice (25.4mm) |
| em | 1.5em | Relativa alla dimensione del font corrente |
| % | 80% | Percentuale del contenitore |
| fr | 1fr | Frazione dello spazio disponibile (solo in grid) |

```
// Operazioni su lunghezze:  
#(2cm + 5mm) // 2.5cm  
#(1in - 1cm) // ~1.546cm  
#(2 * 1.5cm) // 3cm
```

13.7. Colori

I colori si specificano in vari modi:

```
// RGB con valori 0-255  
#rgb(255, 0, 0) // Rosso puro  
#rgb(0, 128, 255) // Blu  
// RGB con stringa esadecimale  
#rgb("#ff0000") // Rosso  
#rgb("#2471a3") // Blu medio  
// Scala di grigi  
#luma(0) // Nero  
#luma(128) // Grigio medio  
#luma(255) // Bianco  
// CMYK (per stampa)  
#cmyk(0%, 100%, 100%, 0%) // Rosso CMYK  
// Colori predefiniti  
color.red
```

```
color.blue
color.green
color.black
color.white
color.gray
color.yellow
color.orange
color.purple
// Modificare un colore
#color.red.darken(30%) // Rosso più scuro
#color.blue.lighten(50%) // Blu più chiaro
#color.green.transparentize(70%) // Verde trasparente
```

13.8. Array (liste)

Un array è una sequenza ordinata di valori:

```
// Creare un array
#let numeri = (1, 2, 3, 4, 5)
#let nomi = ("Mario", "Lucia", "Carlo")
#let misto = (1, "due", true, 3.14)
// Accedere agli elementi (gli indici partono da 0!)
#numeri.at(0) // 1 (primo elemento)
#numeri.at(4) // 5 (quinto elemento)
#numeri.first() // 1
#numeri.last() // 5
// Lunghezza
#numeri.len() // 5
// Aggiungere elementi
#let numeri = (1,2,3,4,5)
#let nuovi = numeri + (6,) // (1,2,3,4,5,6)
//metodo push
#numeri.push(6) // Aggiunge 6 in fondo: ora numeri = (1,2,3,4,5,6).
// push() modifica la variabile sul posto. Gli array hanno semantica
// per copia: passandoli a una funzione, l'originale non viene toccato.
// Slice (sottoarray)
#numeri.slice(1, 3) // (2, 3) – dal secondo al terzo
// Mappare (trasformare ogni elemento)
#numeri.map(n => n * 2) // (2, 4, 6, 8, 10)
// Filtrare
#numeri.filter(n => n > 2) // (3, 4, 5)
// Somma/concatenazione
#(1, 2) + (3, 4) // (1, 2, 3, 4)
// Contiene?
#numeri.contains(3) // true
// Invertire
#numeri.rev() // (5, 4, 3, 2, 1)
```

```
// Ordinare
#(3, 1, 4, 1, 5).sorted() // (1, 1, 3, 4, 5)
```

Nota

In Typst, come in tutti i linguaggi di programmazione seri (vero R?), gli indici degli array **partono da 0**: il primo elemento è `.at(0)`, il secondo `.at(1)`, ecc. Questo è standard in quasi tutti i linguaggi di programmazione moderni.

13.9. Dizionari (dict)

Un dizionario associa **chiavi** (nomi) a **valori**:

```
// Creare un dizionario
#let persona = (
  nome: "Mario",
  età: 25,
  città: "Roma",
)
// Accedere ai valori
#persona.nome // "Mario"
#persona.at("età") // 25 (sintassi alternativa)
// Le chiavi sono stringhe (senza virgolette per comodità)
// Aggiungere o modificare
#let persona = (nome: "Mario", età: 25, città: "Roma")
#let nuova_persona = (..persona, email: "mario@esempio.it")
//metodo insert: aggiunge (o aggiorna) una chiave modificando la variabile sul posto
#persona.insert("email", "mario@esempio.it") // ora persona contiene anche email
// Le chiavi
#persona.keys() // ("nome", "età", "città")
// I valori
#persona.values() // ("Mario", 25, "Roma")
// Contiene una chiave?
#persona.keys().contains("nome") // true
```

13.10. Content

Il tipo `content` è il tipo nativo di Typst per il contenuto del documento. Ogni testo, immagine, tabella... è content:

```
// Creare content con le quadre:
#let testo = [Ciao, *mondo*!]
// Content è diverso da stringa:
#let s = "Ciao" // string
```

```
#let c = [Ciao] // content (potrebbe avere markup)
// Usare content come argomento
#block(fill: yellow)[#testo]
```

13.11. none e auto

```
// none: assenza di valore
#let valore = none
#if valore == none [Non c'è niente]
// auto: valore calcolato automaticamente da Typst
#set page(header: auto) // Typst decide da solo
```

13.12. Scoprire il tipo di un valore

La funzione `type()` ti dice che tipo è un valore:

► La funzione type()

CODICE TYPST

```
Il tipo di 42 è: #type(42)
Il tipo di "ciao" è: #type("ciao")
Il tipo di 3.14 è: #type(3.14)
Il tipo di true è: #type(true)
Il tipo di 2cm è: #type(2cm)
Il tipo di (1,2,3) è: #type((1,2,3))
```

RISULTATO

Il tipo di 42 è: `int` Il tipo di «ciao» è:
`str` Il tipo di 3.14 è: `float` Il tipo di true
è: `bool` Il tipo di 2cm è: `length` Il tipo
di (1,2,3) è: `array`

13.13. Conversioni tra tipi

```
#int(3.7) // 3 (tronca, non arrotonda)
#float(5) // 5.0
#str(42) // "42"
#str(3.14) // "3.14"
#int("42") // 42
```

14. Variabili

Una variabile è un **nome** a cui associ un **valore**. Una volta definita, puoi usare quel nome invece di riscrivere il valore ogni volta.

14.1. Definire una variabile con `#let`

La parola chiave per definire variabili è `let`:

```
#let x = 5
#let nome = "Mario"
#let attivo = true
#let colore-principale = rgb("#1a3a5c")
```

14.2. Usare una variabile

Una volta definita, usi la variabile scrivendo `#nome-variabile`:

► Uso delle variabili

CODICE TYPST

```
#let autore = "Giulia Bianchi"
#let anno = 2026
#let titolo = "La Mia Tesi"
= #titolo
Autore: *#autore* --- Anno: #anno
```

RISULTATO

La Mia Tesi Autore: **Giulia Bianchi** — Anno: 2026

14.3. Perché usare le variabili?

Le variabili sono utili per:

1. **Evitare ripetizioni**: se il nome dell'autore compare in 10 posti, basta cambiarlo una volta nella definizione.
2. **Rendere il codice leggibile**: `colore-accento` è più chiaro di `rgb("#1a3a5c")` ripetuto ovunque.
3. **Riusare contenuto**: puoi definire un blocco di testo e inserirlo più volte.

► Variabili per la configurazione

```
#let colore-titolo = rgb("#1a3a5c")
#let colore-testo = luma(50)
#let spaziatura = 1.5em
#set text(fill: colore-testo)
#set par(leading: spaziatura)
#text(fill: colore-titolo, size: 16pt, weight: "bold")[
  Il mio titolo ben configurato
]
Un paragrafo con la spaziatura impostata sopra.
```

14.4. Scope delle variabili

Le variabili sono disponibili nel punto in cui sono definite e in tutto ciò che viene dopo:

```
#let a = 10
// 'a' è disponibile qui
#{
  let b = 20 // 'b' esiste solo dentro queste graffe
  a + b // OK: a è definita fuori
}
// 'b' NON è disponibile qui (fuori dallo scope)
```

Nota

Una variabile definita fuori da qualsiasi blocco `{ }` o `[ ]` è disponibile per tutto il documento. Una definita dentro un blocco `{ }` esiste solo dentro quel blocco.

14.5. Variabili che contengono contenuto

Una delle cose più potenti: le variabili possono contenere blocchi di contenuto (markup) da riutilizzare:

► Variabile contenuto riusabile

CODICE TYPST

```
#let avviso-sicurezza = [
  *Attenzione:* Prima di procedere,
  assicurati
  di aver salvato il lavoro.
```

RISULTATO

Passaggio 1 Attenzione: Prima di procedere, assicurati di aver salvato il lavoro. Qui descrivo il primo passaggio...

```
1
== Passaggio 1
#avviso-sicurezza
Qui descrivo il primo passaggio...
== Passaggio 2
#avviso-sicurezza
Qui descrivo il secondo passaggio...
```

Passaggio 2 Attenzione: Prima di procedere, assicurati di aver salvato il lavoro. Qui descrivo il secondo passaggio...

14.6. Costanti vs variabili

In Typst non esiste una distinzione formale tra «costante» e «variabile». Per convenzione, i valori che non cambiano si definiscono all’inizio del documento e si usano con nomi descrittivi.

14.7. Operazioni e calcoli

Le variabili numeriche supportano tutte le operazioni aritmetiche:

► Calcoli con variabili

CODICE TYPST

```
#let larghezza = 10cm
#let altezza = 7cm
#let area = larghezza * altezza / (1cm
* 1cm)
Larghezza: #larghezza
Altezza: #altezza
Area: #area cm²
```

RISULTATO

Larghezza: 10cm Altezza: 7cm Area:
70 cm²

15. Condizioni: if, else if, else

Le condizioni ti permettono di mostrare o eseguire cose **solo quando una certa condizione è vera**. Sono uno strumento fondamentale nella programmazione.

15.1. La struttura if di base

```
#if condizione {  
  // Codice eseguito se condizione è true  
}
```

O con contenuto:

```
#if condizione [  
  Contenuto mostrato se la condizione è vera.  
]
```

► if di base

CODICE TYPST

```
#let temperatura = 35  
#if temperatura > 30 [  
  Fa molto caldo oggi!  
]  
#if temperatura < 0 [  
  Attenzione: temperatura sotto zero!  
]
```

RISULTATO

Fa molto caldo oggi!

15.2. if ... else

Per gestire il caso in cui la condizione è falsa:

```
#if condizione [  
  Mostrato se vero.  
] else [  
  Mostrato se falso.  
]
```

► if ... else

CODICE TYPST

```
#let punteggio = 72  
#if punteggio >= 60 [  
  *Promosso!* Punteggio:  
]
```

RISULTATO

Promosso! Punteggio: 72/100.

```
#punteggio/100.  
] else [  
  *Bocciato.* Punteggio:  
#punteggio/100.  
]
```

15.3. if ... else if ... else

Per più casi:

► Condizioni multiple

CODICE TYPST

```
#let voto = 7  
#if voto >= 9 [  
  Eccellente!  
] else if voto >= 7 [  
  Buono.  
] else if voto >= 6 [  
  Sufficiente.  
] else [  
  Insufficiente.  
]
```

RISULTATO

Buono.

15.4. Operatori di confronto

| Operatore | Significato | Esempio |
|--------------------|-------------------|--------------------------|
| <code>==</code> | Uguale | <code>x == 5</code> |
| <code>!=</code> | Diverso | <code>x != 5</code> |
| <code><</code> | Minore | <code>x < 10</code> |
| <code>></code> | Maggiore | <code>x > 0</code> |
| <code><=</code> | Minore o uguale | <code>x <= 100</code> |
| <code>>=</code> | Maggiore o uguale | <code>x >= 18</code> |

15.5. Operatori logici

Puoi combinare più condizioni:

| Operatore | Significato | Esempio |
|------------------|-----------------|-------------------------------------|
| <code>and</code> | Entrambe vere | <code>x > 0 and x < 10</code> |
| <code>or</code> | Almeno una vera | <code>x < 0 or x > 100</code> |
| <code>not</code> | Negazione | <code>not (x == 0)</code> |

► Condizioni combinate

CODICE TYPST

```
#let età = 25
#let ha-patente = true
#if età >= 18 and ha-patente [
  Può guidare.
] else if età >= 18 and not ha-patente [
  È maggiorenne ma non ha la patente.
] else [
  È minorenne.
]
```

RISULTATO

Può guidare.

15.6. L'if come espressione

In Typst, `if` restituisce un valore! Questo significa che puoi usarlo dove normalmente metteresti un valore:

► if come espressione

CODICE TYPST

```
#let dark-mode = true
#let sfondo = if dark-mode
{ rgb("#1a1a2e") } else { white }
#let colore-testo = if dark-mode
{ white } else { black }
#block(fill: sfondo, inset: 1em, radius:
5pt)[
  #text(fill: colore-testo)[
    Questa è una casella in modalità #if
```

RISULTATO

Questa è una casella in modalità
scura else [chiara].

```
dark-mode [scura]
else [chiara].
]
]
```

16. Cicli: for e while

I cicli ti permettono di **ripetere** un'operazione più volte. Sono fondamentali per generare contenuto in modo automatico.

16.1. Il ciclo for

16.1.1. Iterare su un array

La forma più comune: esegui qualcosa **per ogni elemento** di una collezione:

► for su un array

CODICE TYPST

```
#let frutta = ("Mele", "Pere", "Banane",
"Arance")
#for frutto in frutta [
- #frutto
]
```

RISULTATO

- Mele
- Pere
- Banane
- Arance

16.1.2. Iterare con range

`range(n)` genera i numeri da 0 a n-1:

► for con range

CODICE TYPST

RISULTATO

Il numero 0 al quadrato è 0.
Il numero 1 al quadrato è 1.

```
#for i in range(5) [  
  Il numero #i al quadrato è #(i * i).  
]
```

Il numero 2 al quadrato è 4.
Il numero 3 al quadrato è 9.
Il numero 4 al quadrato è 16.

`range(start, end)` genera da `start` a `end - 1`:

► Tavola del 7

CODICE TYPST

```
#for n in range(1, 11) [  
  #n × 7 = #(n * 7) \  
]
```

RISULTATO

1 × 7 = 7
2 × 7 = 14
3 × 7 = 21
4 × 7 = 28
5 × 7 = 35
6 × 7 = 42
7 × 7 = 49
8 × 7 = 56
9 × 7 = 63
10 × 7 = 70

16.1.3. range con step

```
// range(inizio, fine, step: passo)  
#for n in range(0, 20, step: 5) [#n, ]  
// 0, 5, 10, 15,
```

16.1.4. Generare una tabella con for

Ecco un esempio pratico: generare una tabella di moltiplicazione:

► Tabella con ciclo for

```
#let n-max = 5  
#table(  
  columns: n-max + 1,  
  fill: (col, row) => if col == 0 or row == 0 { C.blu }  
  else if calc.odd(col + row) { white }  
  else { rgb("#f4f6f7") },  
  // Prima riga: intestazione
```

```
table.cell(fill: C.blu)[  
  ..range(1, n-max + 1).map(n =>  
    table.cell()[#text(fill: white, weight: "bold")[#n]]  
  ),  
  // Righe dati  
  ..range(1, n-max + 1).map(i =>  
    (  
      table.cell()[#text(fill: white, weight: "bold")[#i]],  
      ..range(1, n-max + 1).map(j => [#(i * j)]),  
    )  
  ).flatten(),  
)
```

16.1.5. Iterare con indice

Per avere sia l'indice che il valore, usa `.enumerate()`:

► enumerate()

CODICE TYPST

```
#let colori = ("Rosso", "Verde", "Blu")  
#for (i, colore) in colori.enumerate() [  
  #(i + 1). #colore  
]
```

RISULTATO

1. Rosso
2. Verde
3. Blu

16.1.6. Il for restituisce un array

Il ciclo `for` raccoglie i risultati in un array. Se non vuoi che restituisca nulla (es. stai solo usando effetti collaterali), puoi ignorarlo:

```
// Il for restituisce un array di risultati:  
#let quadrati = for i in range(1, 6) { i * i }  
// quadrati = (1, 4, 9, 16, 25)  
// Per convertire in contenuto, usa map + join:  
#range(1, 6).map(i => str(i * i)).join(", ")  
// Risultato: 1, 4, 9, 16, 25
```

16.2. Il ciclo while

Il `while` continua finché la condizione è vera:

```
#let i = 0
#while i < 5 {
  i += 1
  str(i) + " "
}
// Risultato: 1 2 3 4 5
```

Attenzione

Con il ciclo `while`, fai **molta** attenzione a non creare un ciclo infinito (una condizione che rimane sempre vera). Typst potrebbe bloccarsi. Assicurati sempre che la condizione prima o poi diventi `false`.

► Potenze di 2 con while

CODICE TYPST

```
#let n = 1
#while n <= 1000 {
  [#n]
  n *= 2
}
```

RISULTATO

1 2 4 8 16 32 64 128 256 512

16.3. break e continue

Dentro un ciclo puoi:

- `break`: uscire dal ciclo immediatamente
- `continue`: saltare all'iterazione successiva

```
// Stampa i numeri da 1 a 10, ma salta il 5 e si ferma all'8
#for i in range(1, 11) {
  if i == 5 { continue } // Salta il 5
  if i == 9 { break } // Si ferma prima del 9
  [#i]
}
// Risultato: 1 2 3 4 6 7 8
```

16.4. Generare contenuto complesso con cicli

Esempio pratico: generare una serie di schede informative:

► Schede generate con ciclo

CODICE TYPST

```
#let lingue = (  
  (nome: "Python", anno: 1991, uso:  
    "Dati e AI"),  
  (nome: "Rust", anno: 2015, uso:  
    "Sistemi"),  
  (nome: "Typst", anno: 2023, uso:  
    "Documenti"),  
)  
#for l in lingue {  
  block(  
    fill: rgb("#eaf4fb"),  
    inset: 0.8em,  
    radius: 5pt,  
    stroke: (left: 3pt + rgb("#2471a3")),  
    below: 0.7em,  
  ) [  
    *#l.nome* (#l.anno) – #l.uso  
  ]  
}
```

RISULTATO

Python (1991) – Dati e AI

Rust (2015) – Sistemi

Typst (2023) – Documenti

17. Funzioni

Una funzione è un blocco di codice a cui dai un nome e che puoi **richiamare** ogni volta che ne hai bisogno, passandogli diversi input per ottenere diversi output. Pensa a una funzione come a una ricetta: le fornisci gli ingredienti (parametri) e lei ti restituisce il piatto (risultato).

17.1. Funzioni predefinite di Typst

Typst ha centinaia di funzioni già pronte. Ne hai già usate molte:

```
#text(size: 14pt, fill: red)[Ciao] // Funzione 'text'  
#image("foto.png", width: 80%) // Funzione 'image'  
#table(columns: 3, ...) // Funzione 'table'  
#upper("ciao") // "CIAO"  
#lower("CIAO") // "ciao"  
#calc.sqrt(16) // 4.0
```

17.2. Definire le proprie funzioni

Usa `let` per definire una funzione personalizzata:

```
#let nome-funzione(param1, param2) = corpo
```

17.2.1. Funzione semplice

► Prima funzione

CODICE TYPST

```
// Funzione che saluta  
#let saluta(nome) = "Ciao, " + nome +  
"!"  
#saluta("Mario")  
#saluta("Lucia")
```

RISULTATO

Ciao, Mario!
Ciao, Lucia!

17.2.2. Funzione con corpo complesso

Per funzioni con più istruzioni, usa le graffe:

► Funzione con logica

CODICE TYPST

```
#let classifica-voto(v) = {  
  if v >= 9 { "Eccellente" }  
  else if v >= 7 { "Buono" }  
  else if v >= 6 { "Sufficiente" }  
  else { "Insufficiente" }  
}  
- Voto 10: #classifica-voto(10)  
- Voto 7.5: #classifica-voto(7.5)  
- Voto 5: #classifica-voto(5)
```

RISULTATO

- Voto 10: Eccellente
- Voto 7.5: Buono
- Voto 5: Insufficiente

17.3. Parametri con nome e valori predefiniti

Puoi dare un valore predefinito ai parametri. Questo li rende **opzionali**, se non li passi, usa il default:

► Parametri con default

CODICE TYPST

```
#let badge(testo, colore:
  rgb("#2471a3"), testo-colore: white) = {
  box(
    fill: colore,
    inset: (x: 0.7em, y: 0.3em),
    radius: 4pt,
  ) [
    #set text(fill: testo-colore, size: 9pt,
      weight: "bold")
    #testo
  ]
}
#badge("NUOVO")
#badge("URGENTE", colore:
  rgb("#c0392b"))
#badge("OK", colore: rgb("#27ae60"))
#badge("Bozza", colore: luma(180),
  testo-colore: luma(50))
```

RISULTATO

NUOVO

URGENTE

OK

Bozza

Quando chiami la funzione:

- `#badge("NUOVO")` → usa i colori di default
- `#badge("URGENTE", colore: red)` → colore esplicito, il resto di default
- L'ordine dei **parametri con nome** non importa

17.4. Funzioni che accettano contenuto

Il modo più idiomatico in Typst per passare contenuto a una funzione è usare le parentesi quadre [...] direttamente dopo la chiamata:

► Funzione che accetta contenuto

CODICE TYPST

```
#let riquadro-colorato(colore, corpo) =
  block(
    fill: colore.lighten(70%),
    stroke: (left: 4pt + colore),
    inset: (x: 1em, y: 0.8em),
    radius: 4pt,
```

RISULTATO

Questo è un riquadro blu con **cor-**sivo e **grassetto**.

Questo è un riquadro rosso.

```
width: 100%,
)[#corpo]
#riquadro-colorato(blue)[
  Questo è un riquadro blu con
  _corsivo_ e *grassetto*.
]
#riquadro-colorato(red)[
  Questo è un riquadro rosso.
]
```

Ed ecco spiegato come ho fatto questi box particolari, con le funzioni definite!

17.4.1. La sintassi speciale con parametro `body`

Per funzioni dove il contenuto è l'argomento principale, Typst permette una sintassi abbreviata. Il **parametro finale** che si chiama `body` (o qualsiasi altro nome che usi come ultimo parametro posizionale) può essere passato con le `[...]` fuori dalle parentesi:

```
#let evidenzia(colore: yellow, body) = highlight(
  fill: colore,
  body,
)
// Uso normale:
#evidenzia(body: [Testo evidenziato])
// Sintassi abbreviata (il body tra []):
#evidenzia[Testo evidenziato]
#evidenzia(colore: orange)[Testo arancione]
```

► Funzione per enfasi nel testo

CODICE TYPST

```
#let enfasi(corpo) = text(
  fill: rgb("#1a5276"),
  style: "italic",
  weight: "bold",
)[#corpo]
Nel testo puoi #enfasi[enfatizzare]
certe parole
in modo #enfasi[coerente] su tutto il
documento.
```

RISULTATO

Nel testo puoi **enfatizzare** certe parole
in modo **coerente** su tutto il documen-
to.

17.5. Funzioni che restituiscono valori

Le funzioni possono restituire qualsiasi tipo: numeri, stringhe, contenuto, array, dizionari...

► Funzione che restituisce un dizionario

CODICE TYPST

```
// Funzione che crea una palette di colori
#let palette(colore-base) = (
  chiaro: colore-base.lighten(60%),
  normale: colore-base,
  scuro: colore-base.darken(30%),
)
#let blu = palette(rgb("#2471a3"))
#block(fill: blu.chiaro, inset: 0.5em)
[Chiaro]
#block(fill: blu.normale, inset: 0.5em)[
  #text(fill: gray)[Normale]
]
#block(fill: blu.scuro, inset: 0.5em)[
  #text(fill: white)[Scuro]
]
```

RISULTATO

Chiaro



[Normale]

Scuro

17.6. Funzioni freccia (arrow functions / closures)

Typst supporta le funzioni «anonime» — funzioni senza nome, definite al volo. Si usano molto con `map`, `filter`, ecc.:

```
// Funzione anonima: parametro => espressione
#let doppio = n => n * 2
#doppio(5) // 10
// Usata inline con map:
#(1, 2, 3, 4, 5).map(n => n * n)
// (1, 4, 9, 16, 25)
// Con più parametri: usa graffe
#let somma = (a, b) => a + b
#somma(3, 4) // 7
```

17.7. Ricorsione

Una funzione può chiamare se stessa (ricorsione). Utile per strutture annidate:

► Funzione ricorsiva**CODICE TYPST**

```
#let fattoriale(n) = {  
  if n <= 1 { 1 }  
  else { n * fattoriale(n - 1) }  
}  
- 5! = #fattoriale(5)  
- 10! = #fattoriale(10)
```

RISULTATO

- 5! = 120
- 10! = 3628800

17.8. Esempio completo: template di carta

Creare un template riusabile per lettere o schede:

► Template carta riusabile**CODICE TYPST**

```
#let carta(titolo: "Titolo", colore: C.blu,  
corpo) = block(  
  width: 100%,  
  radius: 8pt,  
  stroke: 1pt + luma(210),  
  clip: true,  
  below: 1em,  
) [  
  #block(  
    fill: colore,  
    width: 100%,  
    inset: (x: 1em, y: 0.6em),  
  ) [  
    #text(fill: white, weight: "bold")  
    [#titolo]  
  ]  
  #block(  
    fill: white,  
    width: 100%,  
    inset: (x: 1em, y: 0.9em),  
  ) [#corpo]  
]  
#carta(titolo: " Nota Importante") [  
  Ricordati di salvare il file prima di
```

RISULTATO**Nota Importante**

Ricordati di salvare il file prima di chiudere.

Completato

Il documento è stato consegnato con successo.

```
chiudere.  
]  
#carta(titolo: " Completato", colore:  
  rgb("#1e8449"))[  
  Il documento è stato consegnato con  
  successo.  
]
```

18. Le regole set

Le regole `set` sono uno dei meccanismi più potenti e eleganti di Typst. Ti permettono di modificare i **parametri predefiniti** di qualsiasi elemento, una volta per tutte.

18.1. Cos'è una regola set?

Immagina di volere che tutto il testo del tuo documento sia di dimensione 12pt con font «Georgia». Potresti scrivere `#text(size: 12pt, font: "Georgia")` attorno a ogni singolo paragrafo, giusto? Sì, per pederede del tempo...sarebbe una follia, usiamo typst per ottimizzare! Con `set`, dici una sola volta:

```
#set text(size: 12pt, font: "Georgia")
```

E da quel punto in poi, **tutto** il testo usa quelle impostazioni.

Regola set — Una regola `set` modifica i parametri di default di una funzione (o elemento) per tutto ciò che viene dopo. Sintassi: `#set funzione(parametro: valore)`.

18.2. set text — il testo

```
#set text(  
  font: "Linux Libertine", // Nome del font  
  size: 11pt, // Dimensione  
  fill: luma(30), // Colore (quasi nero)  
  weight: "regular", // "bold", "light", "regular", ...  
  style: "normal", // "italic", "oblique"  
  tracking: 0pt, // Spaziatura tra le lettere  
  spacing: 100%, // Spaziatura tra le parole
```

```
lang: "it", // Lingua (per trattini, ecc.)  
hyphenate: true, // Sillabazione automatica  
)
```

► set text

CODICE TYPST

```
#set text(font: "Roboto", size: 13pt, fill:  
rgb("#2c3e50"))  
Questo testo usa Roboto a 13pt in blu  
scuro.  
Tutti i paragrafi che seguono useranno  
questo stile.
```

RISULTATO

Questo testo usa Roboto a 13pt in blu scuro. Tutti i paragrafi che seguono useranno questo stile.

18.3. set par — i paragrafi

```
#set par(  
justify: true, // Giustificazione (allineamento sx e dx)  
leading: 0.75em, // Interlinea (spazio tra le righe)  
spacing: 1.2em, // Spazio tra i paragrafi  
first-line-indent: 1.5em, // Rientro prima riga  
hanging-indent: 0pt, // Rientro sospeso  
)
```

► set par

CODICE TYPST

```
#set par(justify: true, leading: 1.2em,  
first-line-indent: 1.5em)  
Questo è il primo paragrafo con  
giustificazione attivata  
e interlinea aumentata. Noti come il  
testo si distribuisce  
uniformemente su tutta la larghezza  
della colonna.  
Questo è il secondo paragrafo. La  
prima riga ha un rientro  
come in un libro tradizionale.
```

RISULTATO

Questo è il primo paragrafo con giustificazione attivata e interlinea aumentata. Noti come il testo si distribuisce uniformemente su tutta la larghezza della colonna. Questo è il secondo paragrafo. La prima riga ha un rientro come in un libro tradizionale. L'interlinea più ampia rende il testo più leggibile.

L'interlinea più ampia rende
il testo più leggibile.

18.4. set heading – i titoli

```
#set heading(  
  numbering: "1.1.", // Schema di numerazione  
  outlined: true, // Appare nell'indice?  
)
```

18.5. set page – la pagina

```
#set page(  
  paper: "a4", // Formato carta  
  margin: ( // Margini  
    top: 2.5cm,  
    bottom: 2.5cm,  
    left: 3cm,  
    right: 2.5cm,  
  ),  
  numbering: "1", // Formato numero pagina  
  number-align: center, // Posizione numero: center, right, left  
  columns: 1, // Numero di colonne  
  fill: white, // Colore sfondo pagina  
  flipped: false, // Orientamento (true = landscape)  
)
```

Formati carta disponibili:

| Formato | Codice | Formato | Codice |
|---------|--------|---------------|---------------------|
| A0 | "a0" | A5 | "a5" |
| A1 | "a1" | A6 | "a6" |
| A2 | "a2" | US Letter | "us-letter" |
| A3 | "a3" | US Legal | "us-legal" |
| A4 | "a4" | Presentazione | "presentation-16-9" |

18.6. set list e set enum

```
// Lista puntata
#set list(
  marker: "→", // Marcatore personalizzato
  indent: 1.5em, // Rientro
  spacing: 0.6em, // Spazio tra le voci
)
// Lista numerata
#set enum(
  numbering: "1.", // Schema numerazione
  indent: 1.5em,
  spacing: 0.6em,
)
```

18.7. set table – le tabelle

```
#set table(
  stroke: 0.5pt + luma(200), // Bordo di default
  inset: 0.5em, // Padding celle
  align: left, // Allineamento
  fill: (col, row) => // Funzione colore
    if calc.odd(row) { luma(245) } else { white },
)
```

18.8. set figure – le figure

```
#set figure(
  supplement: "Figura", // Testo prima del numero
  gap: 0.8em, // Spazio tra contenuto e didascalia
  numbering: "1", // Schema numerazione
  placement: none, // Posizionamento
)
```

18.9. set math.equation – le equazioni

```
#set math.equation(
  numbering: "(1)", // Le equazioni vengono numerate
  supplement: "Eq.", // Testo nel riferimento
)
```

18.10. Lo scope delle regole set

Le regole `set` hanno uno **scope**: si applicano da dove sono definite in poi, oppure solo dentro il blocco `{ }` o `[]` in cui si trovano.

► Scope delle regole set

CODICE TYPST

```
Testo normale in 11pt.  
#{  
  set text(size: 14pt, fill: red)  
  [Questo testo è grande e rosso.]  
  [Anche questo!]  
}  
Di nuovo testo normale in 11pt.
```

RISULTATO

Testo normale. Questo testo è grande e rosso. Anche questo! Di nuovo testo normale.

18.11. set in funzioni

Puoi usare `set` dentro una funzione per applicare stile solo al contenuto di quella funzione:

► set dentro una funzione

CODICE TYPST

```
#let capitolo-importante(corpo) = {  
  set text(size: 12pt)  
  set par(leading: 1em)  
  block(  
    fill: rgb("#fef9e7"),  
    inset: 1.2em,  
    radius: 6pt,  
    stroke: 0.5pt + luma(210),  
  )[#corpo]  
}  
#capitolo-importante[  
  Questo contenuto ha stile  
  personalizzato interno.  
  Il testo è leggermente più grande e  
  con più interlinea.  
]  
Questo testo è tornato normale.
```

RISULTATO

Questo contenuto ha stile personalizzato. Il testo ha più interlinea e sfondo giallino.

Questo testo è tornato normale.

19. Le regole show

Le regole `show` sono più potenti delle `set`: non cambiano solo i **parametri** di un elemento, ma ne **trasformano completamente l'aspetto**, intercettando l'elemento prima che venga reso nel PDF.

Regola show — Una regola `show` intercetta un elemento (o tutti gli elementi di un certo tipo) e lo trasforma in qualcosa di diverso. Sintassi: `#show selettore: trasformazione`.

19.1. La differenza tra set e show

```
// set: modifica i parametri → cambia proprietà
#set text(size: 14pt) // Il testo diventa più grande
// show: trasforma l'elemento → può cambiare struttura
#show strong: it => text(fill: red, it.body) // Il grassetto diventa rosso
```

19.2. Trasformare il grassetto

► show strong

CODICE TYPST

```
// Rendi il grassetto di colore blu
#show strong: it => text(fill:
  rgb("#1a3a5c"), it.body)
Questo è testo normale.
Questo è *testo in grassetto*, ora
appare blu.
*Anche questa frase* è in grassetto
blu.
```

RISULTATO

Questo è testo normale. Questo è **te-
sto in grassetto**, ora appare blu. Anche
questa frase è in grassetto blu.

19.3. Trasformare i link

► show link

CODICE TYPST

```
#show link: it => {
  set text(fill: rgb("#2471a3"))
  underline(stroke: (paint:
    rgb("#2471a3"), dash: "dashed"), it)
}
Visita #link("https://typst.app")[Typst]
o
#link("https://github.com/typst/typst")
[GitHub].
```

RISULTATO

Visita [Typst](https://typst.app) o [GitHub](https://github.com/typst/typst).

19.4. Trasformare il codice inline

► show raw (codice inline)

CODICE TYPST

```
#show raw.where(block: false): it =>
box(
  fill: rgb("#fff3e0"),
  inset: (x: 4pt, y: 2pt),
  radius: 3pt,
  stroke: 0.5pt + rgb("#ff8f00"),
)[
  #set text(fill: rgb("#e65100"), font:
    "Courier New")
  #it
]
Usa #raw("npm install") per installare i
pacchetti.
La variabile #raw("n") è dichiarata con
#raw("let n = 0").
```

RISULTATO

Usa `npm install` per installare i pacchetti. La variabile `n` è dichiarata con `let n = 0`.

19.5. Selettori: .where()

Puoi applicare la regola `show` solo a un sottoinsieme di elementi usando `.where()`:

```
// Solo i titoli di livello 1
#show heading.where(level: 1): it => ...
```

```
// Solo il codice a blocco (non inline)
#show raw.where(block: true): it => ...
// Solo le figure che contengono immagini
#show figure.where(kind: image): it => ...
// Solo le figure che contengono tabelle
#show figure.where(kind: table): it => ...
```

19.6. Trasformare i titoli

► show heading personalizzato

CODICE TYPST

```
#show heading.where(level: 2): it => {
  v(1em)
  block[
    #set text(fill: rgb("#7d3c98"), size:
    14pt)
    #it.body
    #v(-0.2em)
    #line(length: 100%, stroke: 2pt +
    rgb("#7d3c98"))
  ]
  v(0.4em)
}
== Prima sezione viola
Testo sotto la prima sezione.
== Seconda sezione viola
Testo sotto la seconda sezione.
```

RISULTATO

Prima sezione viola

Testo sotto la prima sezione.

Seconda sezione viola

Testo sotto la seconda sezione.

19.7. Trasformare il documento intero (show globale)

La forma più potente: trasforma il contenuto di tutto il documento wrappandolo in qualcosa:

```
// Metti tutto il documento in una colonna con max-width
#show: body => block(
  width: 80%,
  inset: 2cm,
)[#body]
// Oppure applica impostazioni globali
#show: body => {
  set text(size: 11pt, font: "Linux Libertine")
```

```
set par(justify: true)
body
}
```

Questa è la base di tutti i **template** Typst.

19.8. Trasformare le figure

► show figure personalizzato

```
#show figure.where(kind: image): it => {
  align(center)[
    #block(
      stroke: 2pt + luma(200),
      radius: 6pt,
      clip: true,
      inset: 0pt,
    )[#it.body]
    #v(0.3em)
    #text(size: 9pt, fill: luma(100), style: "italic")[
      #it.supplement #context it.counter.display(): #it.caption.body
    ]
  ]
}
```

19.9. show per sostituzione testo

Puoi usare `show` per sostituire automaticamente stringhe di testo con altro contenuto:

► show per sostituzione testo

CODICE TYPST

```
// Ogni volta che scrivi "Typst",
aggiunge il logo
#show "Typst": [*Typst*#box(
  baseline: -0.25em,
  height: 0.8em,
)]#text(size: 0.6em, fill:
rgb("#239dad"), baseline: 0pt)[™]
```

RISULTATO

Benvenuti nel mondo di **Typst**. **Typst** è un linguaggio potente. Usiamo **Typst** ogni giorno.

```
]]
```

```
Benvenuti nel mondo di Typst.  
Typst è un linguaggio potente.
```

20. Colori e grafica

20.1. Sistemi di colore

Typst supporta diversi modi per specificare i colori.

20.1.1. RGB

Il più comune: tre valori (Rosso, Verde, Blu) da 0 a 255:

```
#rgb(255, 0, 0) // Rosso puro  
#rgb(0, 0, 255) // Blu puro  
#rgb(128, 128, 128) // Grigio medio  
// Con stringa esadecimale (come in CSS):  
#rgb("#ff0000") // Rosso  
#rgb("#1a3a5c") // Blu scuro  
#rgb("#27ae60") // Verde
```

20.1.2. Scala di grigi

```
#luma(0) // Nero  
#luma(128) // Grigio 50%  
#luma(255) // Bianco
```

20.1.3. OKLCH (moderno, perceptually uniform)

```
// oklch(luminosità, croma, tonalità)  
#oklch(60%, 0.2, 240deg) // Blu medio  
#oklch(80%, 0.15, 30deg) // Arancio chiaro
```

20.1.4. Colori predefiniti

black white gray silver
red maroon orange yellow
olive green teal navy
blue aqua purple fuchsia
lime lime ...

Usali con il prefisso `color.` :

`color.red color.blue color.green`
// Oppure direttamente (senza prefisso nei contesti colore):
`red blue green`

20.2. Manipolare i colori

```
// Schiarire
#color.red.lighten(30%) // Rosso più chiaro del 30%
#rgb("#1a3a5c").lighten(50%)
// Scurire
#color.blue.darken(20%)
// Trasparenza (0% = opaco, 100% = invisibile)
#color.green.transparentize(60%)
// Mescolare due colori
#color.mix(red, blue) // Mix 50/50
#color.mix((red, 30%), (blue, 70%)) // Mix pesato
// Invertire
#color.red.negate()
```

► Sfumatura colori

CODICE TYPST

```
#let base = rgb("#2471a3")
#for p in (0%, 20%, 40%, 60%, 80%) {
  block(
    fill: base.lighten(p),
    width: 100%,
    inset: 0.4em,
    radius: 3pt,
    below: 0.3em,
  )
  #text(fill: if p > 50% { black } else
    { white })[
    Schiarito del #p
  ]
}
```

RISULTATO

Schiarito del 0%

Schiarito del 20%

Schiarito del 40%

Schiarito del 60%

Schiarito del 80%

```
}  
}  
}
```

20.3. Gradienti

I gradienti sono transizioni fluide tra due o più colori.

20.3.1. Gradiente lineare

```
gradient.linear(colore1, colore2)  
gradient.linear(colore1, colore2, angle: 45deg)  
gradient.linear(  
  (colore1, 0%), // Colore all'inizio  
  (colore2, 60%), // Colore al 60%  
  (colore3, 100%), // Colore alla fine  
  angle: 135deg,  
)
```

20.3.2. Gradiente radiale

Io ve li faccio vedere, ma non usateli, li odiano tutti.

```
gradient.radial(colore-centro, colore-bordo)  
gradient.radial(  
  white,  
  blue,  
  center: (30%, 40%), // Centro spostato  
  radius: 60%,  
)
```

► Blocco con gradiente

CODICE TYPST

```
#block(  
  width: 100%,  
  height: 3cm,  
  radius: 8pt,  
  fill: gradient.linear(  
    rgb("#1a5276"), rgb("#2980b9"),
```

RISULTATO

Gradiente blu

```
rgb("#aed6f1"),
angle: 135deg,
),
)[
  #align(center + horizon)[
    #text(fill: white, size: 16pt, weight:
"bold")[
  Gradiente blu
  ]
  ]
]
```

20.4. Disegnare forme base

20.4.1. Linea

```
#line(length: 100%, stroke: 1pt + black) // Orizzontale
#line(start: (0pt, 0pt), end: (2cm, 1cm), stroke: red) // Diagonale
```

20.4.2. Rettangolo

```
#rect(
width: 4cm,
height: 2cm,
fill: rgb("#eaf4fb"),
stroke: 1pt + rgb("#2471a3"),
radius: 5pt,
)
```

20.4.3. Cerchio ed ellisse

```
#circle(radius: 1cm, fill: red)
#ellipse(width: 4cm, height: 2cm, fill: blue)
```

► Forme geometriche

CODICE TYPST

RISULTATO

```
#grid(
  columns: (1fr, 1fr, 1fr),
  gutter: 1em,
  align: center,
  rect(width: 100%, height: 1.5cm, fill:
    rgb("#eaf4fb"),
    stroke: 2pt + C.blu2, radius: 8pt),
  circle(radius: 0.75cm, fill:
    rgb("#eafaf1"),
    stroke: 2pt + C.verde2),
  ellipse(width: 100%, height: 1.5cm,
    fill: rgb("#fef5e7"),
    stroke: 2pt + C.arancio),
)
```



Ed è così che ho creato la copertina di questo handbook. Il mio consiglio è di sperimentare un po' con gli hex e gli rgb (online ci sono delle palette stupende, prendete quelle).

20.5. Lo stroke

Lo `stroke` definisce il bordo di una forma. Puoi specificarlo in modo semplice o molto dettagliato:

```
// Semplice (spessore + colore)
stroke: 2pt + black
// Dettagliato
stroke: stroke(
  thickness: 2pt,
  paint: rgb("#1a3a5c"),
  cap: "round", // "butt", "round", "square"
  join: "round", // "miter", "round", "bevel"
  dash: "dashed", // "solid", "dashed", "dotted", "dot-dash"
)
// Solo lato sinistro (per i box)
stroke: (left: 4pt + blue)
// Bordi diversi per ogni lato
stroke: (
  top: 2pt + black,
  bottom: 1pt + gray,
  left: none,
  right: none,
)
```

21. Box e blocchi

`box` e `block` sono i due contenitori fondamentali in Typst. Capire la differenza è essenziale per controllare il layout.

21.1. box vs block: la differenza

| | box | block |
|------------|-------------------------------|-----------------------------------|
| Tipo | Inline (nel flusso del testo) | Block (occupa tutta la larghezza) |
| Uso | Dentro una riga di testo | Elemento separato |
| Dimensione | Si adatta al contenuto | Larghezza 100% di default |
| Va a capo? | No (sta in riga) | Sì (nuovo rigo) |

► box vs block

CODICE TYPST

```
Testo con #box(fill: yellow, inset: 3pt)
[parola evidenziata]
nel mezzo della frase.
#block(fill: rgb("#eaf4fb"), inset: 1em,
radius: 5pt)[
  Questo è un blocco separato che
  occupa
  tutta la larghezza disponibile.
]
Testo che riprende dopo il blocco.
```

RISULTATO

Testo con parola evidenziata nel mezzo della frase.

Questo è un blocco separato che occupa tutta la larghezza disponibile.

Testo che riprende dopo il blocco.

21.2. Parametri di box e block

Entrambi condividono questi parametri principali:

```
#block(
  // Dimensioni
  width: 80%, // Larghezza (length, ratio, o auto)
  height: auto, // Altezza

  // Spaziatura interna
  inset: 1em, // Uguale su tutti i lati
```

```

inset: (x: 1em, y: 0.5em), // Orizzontale e verticale
inset: (top: 0.5em, bottom: 0.5em, left: 1em, right: 1em),

// Spaziatura esterna (solo block)
above: 1em, // Spazio sopra
below: 1em, // Spazio sotto

// Aspetto
fill: white, // Colore sfondo
stroke: 1pt + black, // Bordo
radius: 5pt, // Angoli arrotondati

// Comportamento
clip: false, // Taglia il contenuto che eccede?
breakable: true, // Si spezza tra le pagine? (solo block)
)

```

21.2.1. Angoli arrotondati personalizzati

```

#block(
  radius: (top-left: 10pt, top-right: 10pt,
    bottom-left: 0pt, bottom-right: 0pt),
)

```

21.2.2. Inset dettagliato

```

inset: (top: 0.5em, bottom: 0.5em, left: 1.5em, right: 0.5em)
inset: (x: 1em, y: 0.5em) // Scorciatoia

```

21.3. Impedire le interruzioni di pagina

Se non vuoi che un blocco venga spezzato tra due pagine:

```

#block(breakable: false)[
  Questa tabella importante non deve essere spezzata
  a metà tra due pagine.

  #table(...)
]

```

21.4. Posizionamento con place()

`#place()` posiziona un elemento in modo **assoluto** rispetto al suo contenitore, senza spostare il flusso del testo:

```
#place(
  top + right, // Dove posizionare
  dx: -1cm, // Spostamento orizzontale
  dy: 0.5cm, // Spostamento verticale
  float: false, // true = fluttua nell'area di testo
)[
  #badge("BOZZA")
]
```

Posizioni disponibili:

- Orizzontale: `left`, `center`, `right`
- Verticale: `top`, `horizon`, `bottom`
- Combinazioni: `top + left`, `center + horizon`, ecc.

► Posizionamento con place()

CODICE TYPST

```
#block(width: 100%, height: 3cm,
  fill: rgb("#f4f6f7"), stroke: 1pt +
  luma(200),
  radius: 8pt)[
  #place(top + left, dx: 0.5em, dy:
  0.3em)[
    #text(size: 8pt, fill: luma(120))[In alto
    a sinistra]
  ]
  #place(center + horizon)[
    #text(weight: "bold")[Al centro]
  ]
  #place(bottom + right, dx: -0.5em, dy:
  -0.3em)[
    #text(size: 8pt, fill: luma(120))[In
    basso a destra]
  ]
]
```

RISULTATO

In alto a sinistra

Al centro

In basso a destra

21.5. Stack: impilare elementi

`#stack()` impila elementi uno sopra l'altro o affiancati:

```
// Impila verticalmente (default)
#stack(dir: ttb, spacing: 1em,
  block(fill: red, width: 100%, height: 1cm)[],
  block(fill: green, width: 100%, height: 1cm)[]
)

// Impila orizzontalmente
#stack(dir: ltr, spacing: 0.5em,
  rect(width: 2cm, fill: red),
  rect(width: 3cm, fill: blue),
)
```

21.6. Allineamento

`#align()` allinea il contenuto dentro il suo contenitore:

```
#align(center)[Centrato]
#align(right)[A destra]
#align(left)[A sinistra]
#align(center + top)[Centro in alto]
#align(right + bottom)[Destra in basso]
```

► align() centrato

CODICE TYPST

```
#block(width: 100%, height: 2cm, fill:
  luma(245),
  stroke: 0.5pt + luma(200), radius: 5pt)
[
  #align(center + horizon)[
    Questo testo è centrato
    orizzontalmente e verticalmente.
  ]
]
```

RISULTATO

Questo testo è centrato orizzontalmente e verticalmente.

22. Impaginazione avanzata

22.1. Intestazioni e piè di pagina

Le intestazioni (`header`) e i piè di pagina (`footer`) si definiscono dentro `#set page()`.

22.1.1. Intestazione semplice

```
#set page(  
  header: [  
    Il Mio Documento  
    #h(1fr)  
    Capitolo 1  
  ],  
)
```

22.1.2. Intestazione con numero di pagina

```
#set page(  
  header: context {  
    [Il Mio Libro]  
    h(1fr)  
    counter(page).display()  
  },  
  footer: none,  
)
```

22.1.3. Il contesto (context) e la consapevolezza del documento

Per accedere a informazioni dinamiche del documento (numero di pagina, sezione corrente, contatori) devi usare il blocco `context`. È il modo in cui Typst gestisce le informazioni che cambiano nel documento:

```
#set page(  
  numbering: "1",  
  header: context {  
    // Numero di pagina corrente  
    let pn = counter(page).get().first()  
    // Mostra l'intestazione solo dopo la prima pagina  
    if pn > 1 {  
      grid(  
        columns: (1fr, auto),  
        [Il Mio Documento],  
      )  
    }  
  }  
)
```

```

counter(page).display("1 / 1",
both: true // "pagina corrente / totale pagine"
),
)
line(length: 100%, stroke: 0.5pt + luma(200))
}
},
)

```

22.1.4. Intestazione con titolo sezione corrente

```

#set page(
  header: context {
    let headings = query(heading.where(level: 1).before(here()))
    if headings.len() > 0 {
      let current = headings.last()
      [#current.body_]
      h(1fr)
      counter(page).display()
    }
  },
)

```

22.2. Numerazione pagine

Un po' come per un elenco puntato, ci sono diversi formati di numerazione delle pagine:

| Stringa | Risultato | Uso tipico |
|---------|---------------|----------------------|
| "1" | 1, 2, 3... | Corpo principale |
| "i" | i, ii, iii... | Prefazione/appendice |
| "I" | I, II, III... | Numeri romani |
| "a" | a, b, c... | Appendici |
| "A" | A, B, C... | Appendici maiuscole |
| "1 / 1" | 3 / 12 | Pagina su totale |
| "- 1 -" | - 3 - | Stile rivista |

22.2.1. Cambiare numerazione nel documento

```

// Prefazione: numeri romani
#set page(numbering: "I")
[... prefazione ...]
// Reset e cambio a numeri arabi per il corpo
#counter(page).update(1)

```

```
#set page(numbering: "1")
[... capitoli ...]
```

22.3. Colonne

Per mettere il testo su più colonne:

```
// Due colonne per tutto il documento
#set page(columns: 2)
// Due colonne per un blocco specifico
#columns(2)[
  Questo testo è su due colonne.
  Qui c'è abbastanza testo per vedere
  come funziona il layout a colonne.
]
```

22.3.1. Saltare una colonna

```
#columns(2)[
  Testo nella prima colonna.
  #colbreak() // Salta alla colonna successiva
  Testo nella seconda colonna.
]
```

► Layout a due colonne

CODICE TYPST

```
#columns(2, gutter: 1.5em)[
  == Colonne affiancate
  Questo testo fluisce naturalmente
  nella
  prima colonna. Quando la prima è
  piena,
  il testo continua nella seconda.
  Le colonne sono molto utili per
  bollettini,
  newsletter, o articoli in stile rivista.
  #colbreak()
  == Seconda colonna
  Con #raw("colbreak()") puoi forzare il
  testo
  a saltare immediatamente alla
  colonna successiva,
```

RISULTATO

| Colonne affiancate | Seconda colonna |
|---|---|
| Questo testo fluisce naturalmente nella prima colonna. Quando la prima è piena, il testo continua nella seconda. Le colonne sono molto utili per bollettini, newsletter, o articoli in stile rivista. | Con <code>colbreak()</code> puoi forzare il testo a saltare alla colonna successiva, utile per una nuova sezione. |

```
    utile per iniziare una nuova sezione.  
  ]
```

22.4. Pagine speciali

22.4.1. Pagina con margini diversi

```
// Margini maggiori per la copertina  
#page(  
  margin: 0pt,  
  header: none,  
  footer: none,  
  numbering: none,  
) [  
  // Contenuto della copertina a piena pagina  
]
```

22.4.2. Pagina in landscape (orizzontale)

```
#page(flipped: true) [  
  // Contenuto su pagina orizzontale  
  // Utile per tabelle molto larghe  
]
```

23. Layout a griglia con grid()

`#grid()` è la funzione più versatile per il layout strutturato. Ti permette di disporre elementi in righe e colonne con controllo preciso.

23.1. Sintassi di base

```
#grid(  
  columns: (...), // Definizione colonne  
  rows: (...), // Definizione righe (opzionale)  
  gutter: ..., // Spazio tra celle  
  align: ..., // Allineamento
```

```
[Cella 1], [Cella 2], [Cella 3],  
[Cella 4], [Cella 5], [Cella 6],  
)
```

23.2. Definire le colonne

Identico a `table`:

```
columns: 3 // 3 colonne uguali  
columns: (1fr, 2fr, 1fr) // Proporzioni  
columns: (auto, 1fr) // Prima adattata, seconda riempie  
columns: (3cm, 4cm, 5cm) // Misure fisse
```

23.3. Esempi pratici di layout

23.3.1. Layout copertina (icona + testo)

► Layout icona + testo

CODICE TYPST

```
#grid(  
  columns: (auto, 1fr),  
  gutter: 1em,  
  align: (center + horizon, left),  
  block(  
    fill: C.blu, width: 3cm, height: 3cm,  
    radius: 8pt,  
  ),  
  block[  
    #text(size: 18pt, weight: "bold")[Mio  
Documento]  
    #v(0.3em)  
    #text(fill: luma(100))[Autore: Mario  
Rossi]  
    #v(0.3em)  
    #text(fill: luma(100))[Data: Giugno  
2026]  
  ],  
)
```

RISULTATO



Mio Documen- to

Autore: Mario Rossi

Data: Giugno 2026

23.3.2. Layout a tre colonne informative

► Card a tre colonne

CODICE TYPST

```
#let info-box(titolo, corpo) = block(
  fill: C.sfondo, inset: 1em, radius: 6pt,
  stroke: 1pt + luma(215),
)[
  #align(center)[
    #text(weight: "bold", fill: C.blu)[#titolo]
  ]
  #v(0.5em)
  #corpo
]
#grid(
  columns: (1fr, 1fr, 1fr),
  gutter: 1em,
  info-box("Veloce")[Compilazione
    istantanea],
  info-box("Bello")[Tipografia
    professionale],
  info-box("Potente")[Scripting
    integrato],
)
```

RISULTATO

Veloce

Compi-
lazione
istantanea

Bello

Tipogra-
fia pro-
fessio-
nale

Potente

Scrip-
ting inte-
grato

23.3.3. Grid con celle che si estendono

```
#grid(
  columns: (1fr, 1fr, 1fr),
  gutter: 0.5em,
  // Cella che occupa 2 colonne
  grid.cell(colspan: 2)[Grande cella],
  [Cella normale],
  [A], [B], [C],
)
```

23.4. La funzione layout()

Per layout adattativi che dipendono dalla dimensione disponibile:

```
#layout(size => {
  // 'size' contiene la larghezza e altezza disponibili
  let cols = if size.width > 400pt { 3 } else { 1 }
```

```
grid(columns: cols, ...)  
})
```

24. File multipli e moduli

Quando il documento diventa lungo o vuoi riusare stili in più documenti, conviene organizzarlo su più file.

24.1. `#include` — includere contenuto

`#include "file.typ"` inserisce il **contenuto** di un altro file esattamente dove si trova l'istruzione. È come incollare il contenuto del file nel punto corrente:

```
// Struttura cartelle:  
documento-principale.typ  
capitoli/  
  cap01-introduzione.typ  
  cap02-metodi.typ  
  cap03-risultati.typ  
stili/  
  configurazione.typ
```

```
// documento-principale.typ  
#include "stili/configurazione.typ"  
= Il Mio Documento  
#include "capitoli/cap01-introduzione.typ"  
#include "capitoli/cap02-metodi.typ"  
#include "capitoli/cap03-risultati.typ"
```

24.1.1. Vantaggi di `#include`

- I file sono separati → più facile trovare le cose
- Puoi lavorare su un capitolo alla volta
- I collaboratori lavorano su file diversi → meno conflitti
- Puoi riordinare i capitoli semplicemente spostando le righe

24.2. `#import` — importare funzioni e variabili

`#import` permette di **importare** definizioni (funzioni, variabili, costanti) da un altro file, senza includerne il contenuto testuale:

```
// Nel file 'stili/macro.typ':
#let nota(body) = block(fill: blue.lighten(80%), ...)[#body]
#let avviso(body) = block(fill: red.lighten(80%), ...)[#body]
#let colore-principale = rgb("#1a3a5c")
```

```
// Nel file principale:
#import "stili/macro.typ": nota, avviso, colore-principale
// Ora puoi usare:
#nota[Questa è una nota.]
#avviso[Questo è un avviso.]
```

24.2.1. Importare tutto con *

```
#import "stili/macro.typ": *
// Importa tutte le definizioni pubbliche
```

24.2.2. Importare con alias

```
#import "stili/macro.typ": nota as n, avviso as warn
// Usa #n[] invece di #nota[], e #warn[] invece di #avviso[]
```

24.2.3. Differenza tra import e include

| | #include | #import |
|----------------|----------------------------------|---|
| Cosa fa | Inserisce il contenuto del file | Importa definizioni (funzioni, variabili) |
| Produce testo? | Sì | No |
| Uso tipico | Capitoli, sezioni | Librerie di stile, macro |
| Accede a | Tutto il file viene renderizzato | Solo ciò che esporti |

24.3. Organizzare un progetto grande

Ecco una struttura tipica per una tesi o un libro:

```
tesi/
├── main.typ ← File principale
├── config/
│   ├── pagina.typ ← set page(), set text(), ecc.
│   ├── stili.typ ← show rules, heading styles
│   └── macro.typ ← Funzioni personalizzate
├── capitoli/
│   └── 00-frontespizio.typ
```

```
| |—— 01-introduzione.typ
| |—— 02-teoria.typ
| |—— 03-metodi.typ
| |—— 04-risultati.typ
| |—— 05-conclusioni.typ
| └── appendici.typ
| └── immagini/
|   ├── logo.svg
|   └── grafici/
|     ├── fig01.png
|     └── fig02.png
└── bib/
    └── riferimenti.bib
```

```
// main.typ
#import "config/macro.typ": *
#include "config/pagina.typ"
#include "config/stili.typ"
#include "capitoli/00-frontespizio.typ"
#outline(depth: 3)
#counter(page).update(1)
#include "capitoli/01-introduzione.typ"
#include "capitoli/02-teoria.typ"
// ...
#bibliography("bib/riferimenti.bib")
```

24.4. Variabili condivise tra file

Puoi definire variabili globali in un file di configurazione e importarle ovunque:

```
// config/variabili.typ
#let autore = "Mario Rossi"
#let titolo = "La Mia Tesi"
#let anno = 2026
#let università = "Università degli Studi di Milano"
#let relatore = "Prof. Luigi Verdi"
#let C = ( // Palette colori
  blu: rgb("#1a3a5c"),
  accento: rgb("#2471a3"),
)
```

```
// In qualsiasi capitolo:
#import "config/variabili.typ": autore, anno, C
Tesi di #autore, anno #anno.
```

25. Pacchetti

L'ecosistema di Typst cresce ogni giorno grazie ai pacchetti della comunità. I pacchetti aggiungono funzionalità non presenti nella libreria standard.

25.1. Dove trovare i pacchetti

Il repository ufficiale è **Typst Universe**: <https://typst.app/universe> Puoi cercare per categoria: matematica, presentazioni, CV, tabelle, diagrammi, grafica, citazioni...

25.2. Come importare un pacchetto

```
#import "@preview/nome-pacchetto:versione": funzione1, funzione2
```

Il prefisso `@preview/` indica i pacchetti della comunità.

Nota

Nell'editor online di `typst.app`, i pacchetti vengono scaricati automaticamente. Se usi Typst localmente, vengono scaricati automaticamente la prima volta che li usi e poi messi in cache.

25.3. Pacchetti principali

25.3.1. showybox — Box stilizzate

```
#import "@preview/showybox:2.0.1": showybox
#showybox(
  title: "Teorema di Pitagora",
  frame: (title-color: blue, border-color: blue),
)[
  In un triangolo rettangolo, il quadrato dell'ipotenusa è
  uguale alla somma dei quadrati dei cateti:
  $ a^2 + b^2 = c^2 $
]
```

25.3.2. codly – Codice con numeri di riga

```
#import "@preview/codly:0.2.0": *
#show: codly-init.with()
// Ora i blocchi di codice hanno numeri di riga e stile avanzato
```

25.3.3. cetz – Disegno vettoriale

```
#import "@preview/cetz:0.3.1": canvas, draw
#canvas({
  import draw: *
  circle((0, 0), radius: 1)
  line((0, 0), (1, 0))
  content((0, -1.5), [Cerchio])
})
```

25.3.4. fletcher – Diagrammi e frecce

```
#import "@preview/fletcher:0.5.1" as fletcher: diagram, node, edge
#diagram(
  node((0,0), [Stato A]),
  edge("-", [trasformazione]),
  node((2,0), [Stato B]),
)
```

25.3.5. physica – Notazione fisica

```
#import "@preview/physica:0.9.3": *
$dv(f, x)$ // df/dx
$pdv(f, x, y)$ // Derivata parziale
$laplacian(f)$ // Laplaciano
$grad f$ // Gradiente
$div bold(F)$ // Divergenza
```

25.3.6. polylux – Presentazioni

```
#import "@preview/polylux:0.3.1": *
#set page(paper: "presentation-16-9")
#set text(size: 20pt)
#polylux-slide[
  = Titolo della Slide
  Contenuto della prima slide.
]
#polylux-slide[
  == Seconda Slide
  #pause // Pausa per le animazioni
```

Questo appare dopo la pausa.

]

25.3.7. tablex – Tabelle avanzate

```
#import "@preview/tablex:0.0.8": tablex, cellx
#tablex(
  columns: 3,
  // Celle con colspan/rowspan più semplici
  cellx(colspan: 2)[Unità],
  [Normale],
)
```

25.3.8. lovelace – Pseudocodice e algoritmi

```
#import "@preview/lovelace:0.3.0": *
#pseudocode-list[
  *Input:* Lista  $A$  di  $n$  elementi
  *Output:*  $A$  ordinata
  + *for*  $i$  *from* 1 *to*  $n-1$  *do*
  + *for*  $j$  *from* 0 *to*  $n-i-1$  *do*
  + *if*  $A[j] > A[j+1]$  *then*
  + scambia  $A[j]$  e  $A[j+1]$ 
]
```

25.3.9. codelst – Liste di codice con riferimenti

25.3.10. diagraph – Grafi con GraphViz

```
#import "@preview/diagraph:0.2.5": raw-render
#raw-render(
  digraph {
    A -> B
    B -> C
    A -> C
  })
```

25.4. Come scegliere la versione giusta

Su typst.app/universe ogni pacchetto mostra le versioni disponibili. In generale usa **l'ultima versione stabile** (il numero più alto). Se aggiorni un pacchetto, controlla se ci sono breaking changes nella documentazione.

```
// Sempre specificare la versione esatta:
#import "@preview/showybox:2.0.1": showybox
// Non usare wildcards: "@preview/showybox:*" non funziona
```

26. Documento accademico completo

In questo capitolo costruiamo da zero un template completo per articoli scientifici o tesi di laurea, mettendo insieme tutto quello che abbiamo imparato.

26.1. Template minimo per una tesi

```
// — CONFIGURAZIONE —
#set document(
  title: "Titolo della Tesi",
  author: "Nome Cognome",
)
#set page(
  paper: "a4",
  margin: (top: 3cm, bottom: 3cm, left: 3.5cm, right: 2.5cm),
  numbering: "1",
  number-align: right,
  header: context {
    let pn = counter(page).get().first()
    if pn > 1 {
      set text(size: 9pt, fill: luma(130))
      line(length: 100%, stroke: 0.4pt + luma(200))
      v(-0.8em)
      grid(
        columns: (1fr, auto),
        [ _Titolo della Tesi_ ],
        counter(page).display(),
      )
    }
  },
)
#set text(font: "Linux Libertine", size: 11pt, lang: "it")
#set par(justify: true, leading: 0.75em, spacing: 1.2em)
#set heading(numbering: "1.1.1.")
#set math.equation(numbering: "(1)", supplement: "Eq.")
#set figure(supplement: "Figura")
// — FRONTESPIZIO —
#page(numbering: none, header: none)[
  #align(center)[
```

```
// Logo università (se disponibile)
// #image("logo-univ.svg", width: 4cm)
// #v(1cm)
#text(size: 14pt)[*UNIVERSITÀ DEGLI STUDI DI ...*]
#v(0.3em)
#text(size: 12pt)[Dipartimento di ...]
#v(0.3em)
#text(size: 12pt)[Corso di Laurea in ...]
#v(2cm)
#line(length: 80%, stroke: 1pt + black)
#v(1cm)
#text(size: 11pt, style: "italic")[Tesi di Laurea Magistrale in ...]
#v(0.5cm)
#text(size: 22pt, weight: "bold")[
Titolo della Tesi \
su Più Righe se Necessario
]
#v(0.5cm)
#line(length: 80%, stroke: 1pt + black)
#v(2cm)
#grid(
columns: (1fr, 1fr),
align(left)[
*Relatore:* \
Prof. Luigi Verdi
#v(0.5em)
*Correlatore:* \
Dott. Carlo Bianchi
],
align(right)[
*Candidato:* \
Mario Rossi \
Matricola: 12345678
],
)
#v(1fr)
#text(size: 11pt)[Anno Accademico 2024/2026]
]
]
// — ABSTRACT

#page(numbering: "i", header: none)[
#heading(level: 1, numbering: none, outlined: false)[Abstract]
#text(style: "italic")[
Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Questo è il testo dell'abstract in italiano...
]
#v(1.5em)
*Keywords:* Typst, impaginazione, documenti accademici.
#v(2em)
```

```

#heading(level: 1, numbering: none, outlined: false)[Abstract (English)]
#text(style: "italic")[
  This is the English abstract...
]
#v(1.5em)
*Keywords:* Typst, typesetting, academic documents.
]
// — INDICE

#page(numbering: "i")[
  #outline(depth: 3, indent: 2em)
]
#page(numbering: "i")[
  #outline(target: figure.where(kind: image),
    title: [Lista delle Figure])
  #v(2em)
  #outline(target: figure.where(kind: table),
    title: [Lista delle Tabelle])
]
// — CORPO DEL DOCUMENTO —
#counter(page).update(1)
= Introduzione
Testo del primo capitolo...
== Motivazione
...
== Obiettivi
...
== Struttura del documento
...
= Metodologia
...
= Risultati
...
= Conclusioni e Sviluppi Futuri
...
// — APPENDICI —
#heading(level: 1, numbering: none)[Appendice A — Nome Appendice]
...
// — BIBLIOGRAFIA —
#heading(level: 1, numbering: none, outlined: true)[Bibliografia]
#bibliography("riferimenti.bib", style: "ieee")
// Stili: "ieee", "apa", "chicago-author-date", "mla", ...

```

26.2. Equazioni numerate e riferimenti

```

#set math.equation(numbering: "(1)")
L'equazione di Navier-Stokes per fluidi incompressibili è:

```

```
$ \rho (\partial \mathbf{u}) / (\partial t) +
\rho (\mathbf{u} \cdot \nabla) \mathbf{u} =
-\nabla p + \mu \nabla^2 \mathbf{u} + \mathbf{f}
```

`<eq-navier-stokes>`

Come mostrato nell'equazione `@eq-navier-stokes`, il termine `\rho (\mathbf{u} \cdot \nabla) \mathbf{u}` rappresenta...

26.3. Citazioni bibliografiche

Per citare riferimenti bibliografici, Typst usa il file `.bib` in formato BibTeX:

```
// riferimenti.bib
@article{knuth1984,
  author = {Knuth, Donald E.},
  title = {The \TeX book},
  year = {1984},
  journal = {Computers \& Typesetting},
}
@inproceedings{typst2023,
  author = {Haug, Martin and Mäde, Laurenz},
  title = {Typst: A Programmable Markup Language},
  year = {2023},
}
```

```
// Nel documento:
Come dimostrato da @knuth1984, la qualità tipografica...
L'approccio di Typst @typst2023 migliora su LaTeX...
// In fondo al documento:
#bibliography("riferimenti.bib", style: "ieee")
```

26.4. Figure e tabelle accademiche

```
// Figura con riferimento
#figure(
  image("grafici/risultati.png", width: 85%),
  caption: [
    Confronto delle prestazioni tra i tre algoritmi proposti.
    I valori rappresentano la media su 100 esecuzioni.
  ],
) <fig-risultati>
```

Come mostrato nella `@fig-risultati`, l'algoritmo proposto supera le baseline in tutti i scenari testati.

```
// Tabella con riferimento
#figure(
```

```

table(
  columns: (auto, auto, auto, auto),
  stroke: 0.5pt + luma(200),
  fill: (col, row) => if row == 0 { luma(230) } else { white },
  table.header(
    [*Dataset*], [*Accuratezza*], [*Precisione*], [*Richiamo*],
  ),
  [MNIST], [99.1%], [98.7%], [99.0%],
  [CIFAR-10], [87.3%], [86.9%], [87.1%],
  [ImageNet], [76.4%], [75.8%], [76.2%],
),
caption: [Risultati sperimentali sui dataset di benchmark.],
kind: table,
) <tab-risultati>
La @tab-risultati mostra i risultati quantitativi...

```

27. Template Curriculum Vitae

Un CV moderno e professionale in Typst.

27.1. Template CV a due colonne

```

// — CONFIGURAZIONE —
#set document(title: "CV - Mario Rossi")
#set page(paper: "a4", margin: 0pt)
#set text(font: "Linux Libertine", size: 10pt, lang: "it")
#set par(justify: false, spacing: 0.6em)
// — COLORI —

#let C = (
  sinistra: rgb("#1a3a5c"),
  accento: rgb("#2471a3"),
)
// — FUNZIONI HELPER —

#let voce-cv(titolo, sottotitolo: none, periodo: none, corpo) = {
  block(below: 0.8em)[
    #grid(
      columns: (1fr, auto),
      text(weight: "bold", fill: C.accento)[#titolo],
      if periodo != none { text(fill: luma(120), size: 9pt)[#periodo] },
    )
    #if sottotitolo != none {
      text(style: "italic")[#sottotitolo]
      linebreak()
    }
  ]
}

```

```

}
#corpo
]
}
#let sezione-cv(titolo, corpo) = {
  v(0.5em)
  text(size: 12pt, weight: "bold", fill: C.accento)[#titolo]
  v(-0.3em)
  line(length: 100%, stroke: 1.5pt + C.accento)
  v(0.4em)
  corpo
}
#let competenza(nome, livello) = {
  grid(
    columns: (1fr, auto),
    align: (left + horizon, right + horizon),
    gutter: 0.5em,
    text(size: 9.5pt)[#nome],
    box(width: 4cm)[
      #stack(dir: ltr, spacing: 0pt,
        ..range(5).map(i =>
          box(
            width: 0.7cm, height: 0.35cm,
            radius: 2pt,
            inset: 0pt,
            fill: if i < livello { C.accento } else { luma(220) },
          )
        )
      ],
    )
  )
}
// — LAYOUT: COLONNA SINISTRA (30%) + DESTRA (70%) —————
#grid(
  columns: (30%, 70%),
  // — COLONNA SINISTRA —————
  block(
    fill: C.sinistra,
    height: 100%,
    inset: (x: 1.5em, y: 2em),
    width: 100%,
  )[
    // Foto placeholder (sostituisci con image())
    #align(center)[
      #circle(
        radius: 2cm,
        fill: C.accento,
      )[
        #align(center + horizon)[
          #text(fill: white, size: 28pt)[MR]
        ]
      ]
    ]
  ]
}

```

```

]
]
]
#v(1em)
// Contatti
#set text(fill: white, size: 9.5pt)
#sezione-cv(text(fill: luma(200))[Contatti])[
mario.rossi\@email.it \
+39 333 1234567 \
linkedin.com/in/mariorossi \
Roma, Italia
]
#v(0.5em)
// Competenze tecniche
#sezione-cv(text(fill: luma(200))[Competenze])[
#set text(fill: white)
*Linguaggi*
#competenza("Python", 5)
#competenza("Java", 4)
#competenza("Typst", 5)
#v(0.4em)
*Strumenti*
#competenza("Git", 4)
#competenza("Docker", 3)
#competenza("LaTeX", 4)
]
#v(0.5em)
// Lingue
#sezione-cv(text(fill: luma(200))[Lingue])[
#set text(fill: white)
#competenza("Italiano", 5)
#competenza("Inglese", 4)
#competenza("Tedesco", 2)
]
],
// — COLONNA DESTRA —
block(
fill: white,
height: 100%,
inset: (x: 2em, y: 2em),
width: 100%,
)[
// Nome e titolo
#text(size: 26pt, weight: "bold", fill: C.sinistra)[Mario Rossi]
#linebreak()
#text(size: 13pt, fill: C.accento)[Ingegnere Informatico]
#v(1em)
// Profilo
#sezione-cv("Profilo")[
Ingegnere informatico con 5+ anni di esperienza nello

```

```
sviluppo software. Specializzato in machine learning e
sistemi distribuiti. Appassionato di open-source e
documentazione tecnica con Typst.
]
// Esperienza lavorativa
#sezione-cv("Esperienza Lavorativa")[
#voce-cv(
  "Senior Software Engineer",
  sottotitolo: "Azienda Tech S.p.A. · Roma",
  periodo: "2022 – Presente",
)[
- Sviluppo di sistemi di raccomandazione con Python/PyTorch
- Architettura microservizi su Kubernetes
- Mentoring di 3 junior developer
]
#voce-cv(
  "Software Developer",
  sottotitolo: "Startup Innovazione · Milano",
  periodo: "2019 – 2022",
)[
- Backend API con FastAPI e PostgreSQL
- Riduzione del 40% dei tempi di risposta
]
]
// Formazione
#sezione-cv("Formazione")[
#voce-cv(
  "Laurea Magistrale in Ingegneria Informatica",
  sottotitolo: "Università degli Studi di Roma La Sapienza",
  periodo: "2017 – 2019",
)[
Voto: 110/110 con lode. Tesi: "Machine Learning per
la predizione di guasti industriali".
]
#voce-cv(
  "Laurea Triennale in Informatica",
  sottotitolo: "Università degli Studi di Roma La Sapienza",
  periodo: "2014 – 2017",
)[Voto: 108/110.]
]
// Certificazioni
#sezione-cv("Certificazioni")[
- AWS Certified Solutions Architect (2023)
- Google Professional Data Engineer (2022)
- Scrum Master Certified (2021)
]
],
)
```

27.2. Variante CV minimalista (una colonna)

```
#set page(paper: "a4", margin: (x: 2.5cm, y: 2.5cm))
#set text(font: "Linux Libertine", size: 11pt)
// Header
#align(center)[
  #text(size: 24pt, weight: "bold")[Mario Rossi]
  #linebreak()
  #text(fill: luma(100))[
    mario@email.it · +39 333 1234567 · Roma, Italia ·
    linkedin.com/in/mariorossi
  ]
]
#line(length: 100%, stroke: 0.5pt)
// Sezioni
= Esperienza
...
= Formazione
...
```

28. Presentazioni (Slide)

Typst può creare presentazioni professionali (avete presente beamer in latex?) grazie al pacchetto **Polylux** (o al più recente **Touying**).

28.1. Setup con Polylux

```
// — IMPORTAZIONE —
#import "@preview/polylux:0.3.1": *
// — CONFIGURAZIONE PAGINA —
#set page(
  paper: "presentation-16-9",
  margin: 0pt,
)
#set text(font: "Linux Libertine", size: 22pt, lang: "it")
// — COLORI —

#let C = (
  sfondo: rgb("#0d2137"),
  accento: rgb("#2471a3"),
  testo: white,
  grigio: rgb("#aab7b8"),
)
```

```
// — TEMPLATE SLIDE —
#let slide-base(corpo) = {
  set page(fill: C.sfondo)
  set text(fill: C.testo)
  polylux-slide[
    #block(
      width: 100%, height: 100%,
      inset: (x: 2cm, y: 1.5cm),
    )[#corpo]
  ]
}

#let slide-titolo(titolo, sottotitolo: none, autore: none) = {
  set page(fill: C.sfondo)
  polylux-slide[
    #align(center + horizon)[
      #block(width: 80%)[
        #if sottotitolo != none {
          text(fill: C.accento, size: 16pt, tracking: 3pt)[
            #upper(sottotitolo)
          ]
          v(0.5em)
        }
        text(size: 40pt, weight: "bold")[#titolo]
        v(1em)
        line(length: 40%, stroke: 2pt + C.accento)
        if autore != none {
          v(0.8em)
          text(fill: C.grigio, size: 18pt)[#autore]
        }
      ]
    ]
  ]
}

#let slide-sezione(titolo) = {
  set page(fill: C.accento)
  polylux-slide[
    #align(center + horizon)[
      #text(size: 36pt, weight: "bold", fill: white)[#titolo]
    ]
  ]
}

// — SLIDE DI TITOLO —
#slide-titolo(
  "Introduzione a Typst",
  sottotitolo: "Corso di Typesetting Moderno",
  autore: "Mario Rossi · Giugno 2026",
)

// — SLIDE NORMALE —
#slide-base[
  == Cos'è Typst?
]
```

```

#v(0.5em)
- Un linguaggio di markup _moderno_
- Compilazione *istantanea*
- Qualità tipografica professionale
- Scripting integrato
#align(right)[
#text(fill: C.grigio, size: 16pt)[typst.app]
]
]

// — SLIDE CON COLONNE —
#slide-base[
  == Typst vs LaTeX
  #grid(
    columns: (1fr, 1fr),
    gutter: 2em,
    block(
      fill: white.transparentize(90%),
      inset: 1em, radius: 8pt,
    )[
      *LaTeX*
      - Vecchio (1984)
      - Compilazione lenta
      - Errori criptici
    ],
    block(
      fill: C.accento.transparentize(50%),
      inset: 1em, radius: 8pt,
    )[
      *Typst*
      - Moderno (2023)
      - Istantaneo
      - Errori chiari
    ],
  )
]

// — SLIDE CON RIVELAZIONE PROGRESSIVA —
#slide-base[
  == I vantaggi principali
  #uncover("1-")[Velocità di compilazione]
  #v(0.5em)
  #uncover("2-")[Tipografia eccellente]
  #v(0.5em)
  #uncover("3-")[Scripting potente]
  #v(0.5em)
  #uncover("4-")[Pacchetti della comunità]
]

// — SLIDE CON CODICE —
#slide-base[
  == Il primo documento
  // Dentro una slide scrivi normale markup Typst:

```

= Ciao, Typst!

Questo è il mio primo documento.

- Facile
- Potente
- Bello

`#text`(fill: C.accento)[→ Subito un PDF professionale!]

]

// — SLIDE CONCLUSIVA —

`#slide-base`[

`#align`(center + horizon)[

`#text`(size: 32pt, weight: "bold")[Grazie!]

`#v`(1em)

`#text`(fill: C.grigio, size: 18pt)[

typst.app/universe \ github.com/typst/typst]

`#v`(1em)]

28.2. Rivelazione progressiva (animazioni)

Polylux supporta slide con elementi che appaiono gradualmente:

```
// Mostra solo nella slide 2 e oltre
#only("2-")[Testo che appare dalla slide 2]
// Mostra solo nella slide 3
#only(3)[Testo che appare solo nella slide 3]
// Copre (nasconde) fino alla slide 2
#uncover("2-")[Nascosto, poi rivelato]
// Alternativa: #pause aggiunge una slide di pausa
== Titolo slide
Primo punto.
#pause
Secondo punto (appare dopo click).
#pause
Terzo punto.
```

28.3. Il pacchetto Touying (alternativa avanzata)

Touying è un'alternativa più recente e potente a Polylux:

```
#import "@preview/touying:0.5.3": *
#import themes.university: *
#show: university-theme.with(
  aspect-ratio: "16-9",
  config-info(
    title: [Il Titolo della Presentazione],
    author: [Mario Rossi],
```

```

institution: [Università di ...],
),
)
#title-slide()
= Prima Sezione
== Slide 1
Contenuto della prima slide.
== Slide 2
Contenuto della seconda.

```

Touying offre:

- Molti temi predefiniti (metropolis, dewdrop, university, ...)
- Transizioni più sofisticate
- Handout mode automatico
- Migliore supporto per contenuti complessi

29. Argomenti avanzati

29.1. Il sistema context

Il blocco `context` è il meccanismo con cui Typst gestisce informazioni che dipendono dalla posizione nel documento.

29.1.1. Contatori

Typst ha contatori per pagine, titoli, figure, equazioni... Puoi anche crearne di personalizzati:

```

// Contatore predefinito: pagine
context counter(page).get() // Valore attuale
context counter(page).display() // Visualizzato come testo
// Contatore dei titoli
context counter(heading).get()
// Contatore personalizzato
#let cnt-esempi = counter("esempi")
#cnt-esempi.step() // Incrementa
#context cnt-esempi.display() // Mostra il valore
// Esempio: numerare gli esempi
#let es-numerato(corpo) = {
  cnt-esempi.step()
  block(fill: luma(245), inset: 1em)[
    *Esempio #context cnt-esempi.display()*
  ]
  #corpo
}

```

```
]
}
```

29.1.2. Query

`query()` inside `context` ti permette di cercare elementi nel documento:

```
// Trova tutti i titoli di livello 1
context {
  let capp = query(heading.where(level: 1))
  for cap in capp {
    [• #cap.body #linebreak()]
  }
}

// Trova tutti i titoli prima della posizione corrente
context query(heading.where(level: 1).before(here()))

// Quante figure ci sono nel documento?
context query(figure.where(kind: image)).len()
```

29.1.3. State (stato globale)

Lo stato è come una variabile globale che può essere modificata durante la compilazione:

```
#let tema = state("tema", "chiaro") // Default: "chiaro"
// Modificare lo stato
#tema.update("scuro")
// Leggere lo stato
#context {
  let t = tema.get()
  if t == "scuro" { block(fill: black)[...] }
  else { block(fill: white)[...] }
}
```

29.2. Algoritmi di testo avanzati

29.2.1. Testo con line-by-line control

```
// Controllo fine del testo
#set text(
  tracking: 0.5pt, // Spaziatura tra le lettere (letterspacing)
  spacing: 110%, // Spaziatura tra le parole
)
// Testo con font features
#set text(
```

```
features: ("onum"), // Old-style numerals
)
```

29.2.2. Testo verticale

```
// Ruotare il testo
#rotate(90deg)[Testo verticale]
// Scala il testo
#scale(x: 150%)[Testo allargato]
```

29.3. Pattern e ripetizioni

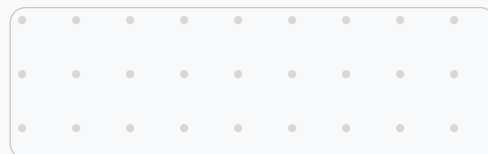
```
// Filigrana (watermark)
#page(background:
  place(center + horizon, rotate(-30deg,
    text(size: 80pt, fill: luma(220))[BOZZA]
  ))
)[
  // Contenuto della pagina
]
```

► Sfondo con pattern

CODICE TYPST

```
// Sfondo con tiling (nelle versioni
vecchie si chiamava `pattern`)
#block(
  width: 100%,
  height: 2.5cm,
  fill: tiling(
    size: (30pt, 30pt),
    relative: "parent",
  )[
    #place(dx: 0pt, dy: 0pt)[
      #circle(radius: 2pt, fill: luma(220))
    ]
  ],
)
```

RISULTATO



29.4. Disegno avanzato con CeTZ

Per diagrammi e disegni vettoriali complessi, usa il pacchetto `cetz`:

```
#import "@preview/cetz:0.3.1": canvas, draw
#canvas({
  import draw: *
  // Assi
  line((-3, 0), (3, 0), mark: (end: "stealth"))
  line((0, -0.5), (0, 3), mark: (end: "stealth"))
  content((3.2, 0), $x$)
  content((0, 3.2), $y$)
  // Parabola tramite punti
  bezier-through(
    (-2, 4), (0, 0), (2, 4),
    stroke: blue + 2pt,
  )
  // Label
  content((1.5, 2.5), [$y = x^2$], anchor: "west")
  // Punti
  for x in (-2, -1, 0, 1, 2) {
    circle((x, x*x), radius: 0.07, fill: blue)
  }
})
```

29.5. Diagrammi con Fletcher

Fletcher è la soluzione elegante per grafi e diagrammi freccia:

```
#import "@preview/fletcher:0.5.1" as fletcher: diagram, node, edge
#diagram(
  cell-size: 15mm,
  node((0,0), [*Compilatore*\nTypst], shape: rect),
  node((2,0), [*PDF*], shape: rect, fill: green.lighten(70%)),
  node((0,1), [.typ\nfile], fill: blue.lighten(80%)),
  edge((0,1), (0,0), "->", [legge]),
  edge((0,0), (2,0), "=>", [produce], label-pos: 0.5),
)
```

Appendice A — Errori Comuni

29.6. Errori di sintassi frequenti

| Errore | Sbagliato | Corretto |
|---------------------------|--|---|
| Apice con più caratteri | <code>\$x^n+1\$</code> | <code>\$x^{(n+1)}\$</code> |
| Frazione | <code>\$a/b+c\$</code> | <code>\$(a/b)+c\$</code> o <code>\$\frac{a}{b+c}\$</code> |
| Stringa con singole | <code>'ciao'</code> | <code>"ciao"</code> |
| Confronto vs assegnazione | <code>if x = 5</code> | <code>if x == 5</code> |
| show rule infinita | <code>show heading: it => heading[#it]</code> | <code>show heading: it => block[#it.body]</code> |
| Chiave dict con spazi | <code>(mia chiave: v)</code> | <code>("mia chiave": v)</code> |
| Accesso a indice fuori | <code>.at(10)</code> (su array di 5) | Controlla <code>.len()</code> prima |
| Tab invece di spazi | Usa tab per indentare liste | Usa 2 spazi |

29.7. Messaggi di errore comuni

| Messaggio | Causa e soluzione |
|---------------------------------|--|
| <code>unknown variable</code> | Hai usato una variabile non definita. Controlla il nome e che sia definita prima dell'uso. |
| <code>expected content</code> | Hai passato un valore sbagliato dove serve contenuto <code>[...]</code> . Metti il testo tra quadre. |
| <code>unexpected keyword</code> | Errore di sintassi. Controlla le virgole, le parentesi e la struttura del codice. |
| <code>file not found</code> | Il file immagine o incluso non esiste nel percorso specificato. Controlla il path. |
| <code>cyclic import</code> | Due file si importano a vicenda. Ristruttura le dipendenze. |
| <code>stack overflow</code> | Ricorsione infinita. Assicurati che la funzione ricorsiva abbia un caso base. |

Appendice B — Risorse e Community

29.8. Documentazione ufficiale

- **Documentazione Typst:** typst.app/docs
- **Typst Universe** (pacchetti): typst.app/universe
- **GitHub Typst:** github.com/typst/typst

29.9. Community e supporto

- **Forum ufficiale:** forum.typst.app
- **Discord Typst:** Cerca «Typst Discord» su Discord per entrare nel server ufficiale
- **Reddit:** reddit.com/r/typst
- **Stack Overflow:** Tag `typst` su StackOverflow per domande tecniche

29.10. Strumenti complementari

- **Tinymist** (VS Code): la migliore estensione per Typst
- **typst-lsp:** Language Server Protocol per qualsiasi editor
- **Hayagriva:** gestione bibliografie in formato YAML
- **typst-fmt:** formatter automatico del codice Typst

Appendice C — Font e Tipografia

29.11. Come specificare i font

```
// Font singolo
#set text(font: "Linux Libertine")
// Lista di fallback: usa il primo disponibile
#set text(font: ("Linux Libertine", "Georgia", "Times New Roman"))
// Font diversi per serif e sans-serif
#set text(font: "Linux Libertine") // Per il corpo
#set raw(theme: "halcyon") // Per il codice
```

29.12. Font disponibili nell'editor online

Nell'editor di typst.app sono disponibili molti font di Google Fonts, tra cui:

| Famiglia | Tipo | Uso consigliato |
|---------------------|------------|-------------------------|
| Linux Libertine | Serif | Corpo testo, accademico |
| New Computer Modern | Serif | Matematica, scientifico |
| Libertinus Serif | Serif | Alternativa a Libertine |
| Source Serif 4 | Serif | Corpo testo leggibile |
| Noto Serif | Serif | Multilingue |
| Linux Biolinum | Sans-serif | Titoli, interfacce |
| Noto Sans | Sans-serif | Presentazioni |
| Source Sans Pro | Sans-serif | Documentazione tecnica |
| Roboto | Sans-serif | Moderno, UI |
| Fira Code | Monospace | Codice sorgente |
| Cascadia Code | Monospace | Codice sorgente |
| JetBrains Mono | Monospace | Codice sorgente |

29.13. Pesì e stili del font

```
// Pesì disponibili (dipende dal font)
#text(weight: "thin")[Thin]
#text(weight: "extralight")[ExtraLight]
#text(weight: "light")[Light]
#text(weight: "regular")[Regular]
#text(weight: "medium")[Medium]
#text(weight: "semibold")[Semibold]
#text(weight: "bold")[Bold]
#text(weight: "extrabold")[ExtraBold]
#text(weight: "black")[Black]
// Peso numerico (100–900)
#text(weight: 350)[Peso 350]
#text(weight: 700)[Peso 700]
// Stili
#text(style: "normal")[Normale]
#text(style: "italic")[Corsivo]
#text(style: "oblique")[Obliquo]
```

29.14. Misure tipografiche

```
// em: relativa al font corrente (1em = dimensione testo)
// 1em di un testo 12pt = 12pt
#set par(leading: 0.75em) // Interlinea 75% della dimensione
// rem: relativa al font di base del documento
// Uguale a em se non sei in un contesto annidato
// ex: altezza della lettera 'x' (circa 0.5em)
// Regola aurea per la tipografia:
// - Corpo testo: 10–12pt per A4
// - Interlinea: 1.2–1.5× il corpo
// - Margini: almeno 2cm, idealmente 2.5–3cm
// - Lunghezza riga: 60–75 caratteri per riga
```

29.15. Highlight e enfasi senza grassetto

```
// Evidenziazione (come pennarello)
#highlight(fill: yellow)[testo evidenziato]
#highlight(fill: rgb("#a8d5ff"))[evidenziato in blu]
// Sottolineato
#underline[testo sottolineato]
#underline(stroke: 2pt + red)[sottolineato rosso spesso]
#underline(stroke: (dash: "dashed"))[sottolineato tratteggiato]
#underline(offset: 3pt)[sottolineato più basso]
// Sopralineato
#overline[testo sopralineato]
// Barrato
#strike[testo barrato]
#strike(stroke: 2pt)[barrato spesso]
```

► Effetti tipografici

CODICE TYPST

Puoi `#highlight(fill: rgb("#fff176"))`
[evidenziare] il testo,
`#underline`[sottolinearlo], o anche
`#strike`[cancellarlo] con stile.
Con `#text(tracking: 3pt, weight: "light")`
[spaziatura delle lettere]
si crea un effetto elegante per i titoli.

RISULTATO

Puoi **evidenziare** il testo, sottolinearlo,
o anche ~~cancellarlo~~ con stile. Con
spaziatura delle lettere
si crea un effetto elegante per i titoli.

Appendice D — Contatori e Numerazione

29.16. Contatori predefiniti

```
// Pagina corrente
context counter(page).get().first() // Numero (int)
context counter(page).display() // Stringa formattata
// Totale pagine
context counter(page).final().first() // Totale pagine
// Stile x/y
context counter(page).display("1 / 1", both: true)
// Contatore titoli
context counter(heading).get() // Array (1, 2, 1) → cap 1.2.1
context counter(heading).display("1.1.")
// Contatore figure
context counter(figure.where(kind: image)).display()
// Contatore equazioni
context counter(math.equation).display()
```

29.17. Creare un contatore personalizzato

Molto utile per numerare esempi, esercizi, teoremi, ecc.:

► Contatore teoremi

CODICE TYPST

```
#let cnt-teorema = counter("teorema")
#let teorema(nome, body) = {
  cnt-teorema.step()
  block(
    fill: rgb("#eaf4fb"),
    stroke: (left: 4pt + C.blu2),
    inset: (x: 1em, y: 0.8em),
    radius: 4pt,
    width: 100%,
  ) [
    *#text(fill: C.blu2)[Teorema #context
cnt-teorema.display()
#if nome != "" [– #nome]]*
```

RISULTATO

Teorema 1 — Pitagora

In ogni triangolo rettangolo, $a^2 + b^2 = c^2$.

Teorema 2

Ogni numero pari > 2 è somma di due primi.

```

#parbreak()
#body
]
}
#teorema("Pitagora")[
  In ogni triangolo rettangolo, il
  quadrato
  dell'ipotenusa è uguale alla somma
  dei
  quadrati dei cateti:  $a^2 + b^2 = c^2$ .
]
#teorema("")[
  Ogni numero pari maggiore di 2 è la
  somma
  di due numeri primi (Congettura di
  Goldbach).
]

```

29.18. Resetare un contatore

```

// Reset a 0
#cnt-teorema.update(0)
// Reset a un valore specifico
#counter(page).update(5) // La pagina successiva sarà la 6
// Reset all'inizio di ogni capitolo
#show heading.where(level: 1): it => {
  cnt-teorema.update(0)
  it
}

```

29.19. Visualizzare il numero totale di pagine

Una richiesta comune: mostrare «Pag. X di Y»:

```

#set page(
  footer: context [
    #h(1fr)
    Pag. #counter(page).display() di
    #counter(page).final().first()
  ]
)

```

Appendice E — Template Pronti all'Uso

Di seguito trovi configurazioni complete e pronte per vari scenari comuni. Copia, incolla, e personalizza (non ringraziatemi).

29.20. Template: Documento generico professionale

```
#set document(title: "Titolo", author: "Nome Cognome")
#set page(paper: "a4", margin: (x: 2.5cm, y: 3cm),
  numbering: "1", number-align: center)
#set text(font: ("Linux Libertine", "Georgia"),
  size: 11pt, lang: "it", hyphenate: true)
#set par(justify: true, leading: 0.8em, spacing: 1.2em)
#set heading(numbering: "1.1.")
#show heading.where(level: 1): set text(size: 18pt)
#show heading.where(level: 2): set text(size: 14pt)
#show link: set text(fill: blue)
#show raw: set text(font: ("Fira Code", "Courier New"), size: 9.5pt)
```

29.21. Template: Documento minimalista

```
#set page(paper: "a4", margin: 3cm, numbering: "1")
#set text(font: "Linux Libertine", size: 12pt, lang: "it")
#set par(justify: true, leading: 0.9em)
#show heading: set text(font: "Linux Biolinum")
```

29.22. Template: Documento tecnico (stile manuale)

```
#set document(title: "Manuale Tecnico", author: "Team Dev")
#set page(
  paper: "a4",
  margin: (left: 3.5cm, right: 2.5cm, top: 2.5cm, bottom: 2.5cm),
  numbering: "1",
  header: context {
    if counter(page).get().first() > 1 {
      grid(columns: (1fr, auto),
        text(size: 8.5pt, fill: luma(150))[_Manuale Tecnico_],
        text(size: 8.5pt, fill: luma(150))[Pag. #counter(page).display()],
      )
    }
  }
)
```

```

v(-0.5em)
line(length: 100%, stroke: 0.4pt + luma(210))
}
},
)
#set text(font: ("Linux Libertine", "Georgia"), size: 11pt, lang: "it")
#set par(justify: true)
#set heading(numbering: "1.1.")
#show raw.where(block: true): it => block(
  fill: luma(245), inset: 1em, radius: 5pt,
  stroke: 0.5pt + luma(200), width: 100%,
)[#set text(font: ("Fira Code", "Courier New"), size: 9.5pt); #it]

```

29.23. Template: Dispensa universitaria

```

#set document(title: "Dispensa", author: "Prof. Cognome")
#set page(paper: "a4",
  margin: (x: 2.5cm, y: 3cm),
  numbering: "1",
  header: context {
    let p = counter(page).get().first()
    if p > 1 {
      line(length: 100%, stroke: 0.4pt + luma(220))
      v(-0.6em)
      align(right, text(size: 8.5pt, fill: luma(130))["
        Corso di Laurea in ... · #p
      "])
    }
  })
#set text(font: ("Linux Libertine", "Georgia"), size: 11pt, lang: "it")
#set par(justify: true, leading: 0.8em)
#set heading(numbering: "1.")
#set math.equation(numbering: "(1)", supplement: "Eq.")
#set figure(supplement: "Figura")
// Box teorema, definizione, esempio
#let teorema(titolo: "Teorema", body) = block(
  fill: rgb("#eaf4fb"), stroke: (left: 4pt + blue),
  inset: 1em, radius: 4pt, width: 100%,
)[*#titolo.* #body]
#let definizione-d(titolo: "Definizione", body) = block(
  fill: rgb("#eafaf1"), stroke: (left: 4pt + green),
  inset: 1em, radius: 4pt, width: 100%,
)[*#titolo.* #body]
#let esempio-d(body) = block(
  fill: rgb("#fef9e7"), stroke: (left: 4pt + orange),
  inset: 1em, radius: 4pt, width: 100%,
)[*Esempio.* #body]

```

29.24. Template: Newsletter / Bollettino

```
#set document(title: "Newsletter")
#set page(paper: "a4", margin: (x: 1.5cm, y: 2cm),
  columns: 2, numbering: none)
#set text(font: ("Linux Libertine", "Georgia"), size: 10pt)
#set par(justify: true, leading: 0.7em)
// Titolo a piena larghezza (fuori dalle colonne)
// Usa una pagina iniziale senza colonne per l'header:
#page(columns: 1)[
  #align(center)[
    #block(fill: rgb("#1a3a5c"), width: 100%,
      inset: (x: 2em, y: 1.5em))[
      #text(fill: white, size: 28pt, weight: "bold")[
        LA MIA NEWSLETTER
      ]
    #linebreak()
    #text(fill: rgb("#aed6f1"), size: 11pt)[
      Volume 1 · Giugno 2026
    ]
  ]
]
#v(0.5em)
]
```

// Da qui il testo va su 2 colonne

29.25. Template: Poster accademico (A1)

```
#set document(title: "Poster")
#set page(
  paper: "a1",
  flipped: true,
  margin: 1.5cm,
)
#set text(font: ("Linux Libertine", "Georgia"), lang: "it")
// Suddividi in 3 colonne con grid
#grid(
  columns: (1fr, 2fr, 1fr),
  gutter: 1cm,
  // Colonna sinistra: metodi
  block(width: 100%)[
    #heading(level: 2, numbering: none)[Metodi]
    ...
  ],
  // Colonna centrale: risultati principali
  block(width: 100%)[
```

```
#heading(level: 2, numbering: none)[Risultati]
...
],
// Colonna destra: conclusioni
block(width: 100%)[
#heading(level: 2, numbering: none)[Conclusioni]
...
],
)
```

Appendice F (Parte I) — Note Aggiuntive: Slide e Presentazioni

Qualche piccola nota aggiuntiva sulle presentazioni di cui abbiamo già parlato.

29.26. Panoramica degli strumenti

| Strumento | Descrizione | Linguaggio | Output |
|-------------------|---|--------------|------------------------|
| Beamer | Pacchetto LaTeX per slide accademiche | LaTeX | PDF |
| R Markdown | Documento computazionale con slide | R + Markdown | PDF / HTML |
| Quarto | Successore moderno di R Markdown | R / Python | PDF / HTML / Reveal.js |
| knitr | Engine di calcolo per R Markdown/Quarto | R | qualsiasi |
| Polylux | Pacchetto Typst per presentazioni | Typst | PDF |
| Touying | Framework Typst avanzato per slide | Typst | PDF |

29.26.1. Beamer

Beamer è il pacchetto LaTeX per presentazioni più usato in ambito accademico e scientifico dal 2003. Produce PDF di alta qualità tipografica, con equazioni perfette e temi predefiniti (Warsaw, Madrid, Metropolis, ecc.).

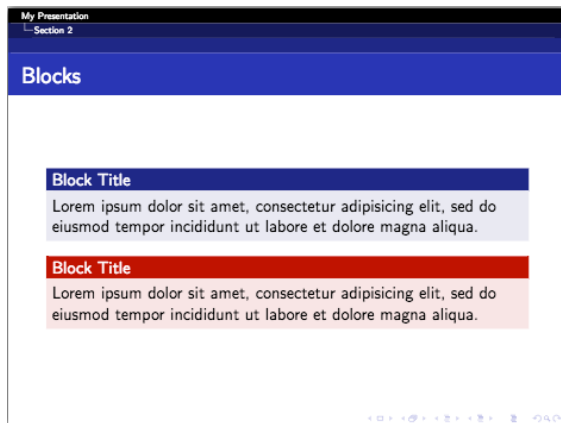


Figura 1: Screenshot di esempio preso da [QUI](#).

La sintassi base di una slide Beamer:

```
\documentclass{beamer}
\usetheme{metropolis}

\begin{document}

\begin{frame}{Titolo della Slide}
\begin{itemize}
\item Prima voce
\pause
\item Seconda voce (appare dopo click)
\end{itemize}
\end{frame}

\end{document}
```

Punti di forza

- Qualità tipografica eccellente
- Temi predefiniti maturi
- Equazioni LaTeX native
- Molto diffuso in ambito accademico

Limiti

- Compilazione lenta (più passaggi)
- Sintassi verbosa e prolissa
- Difficile personalizzare i temi
- Nessun preview in tempo reale

29.26.2. R Markdown

R Markdown è un formato che intreccia testo, codice R e output (grafici, tabelle, risultati) in un unico documento. Può produrre slide tramite il pacchetto `ioslides`, `revealjs` o `beamer`.

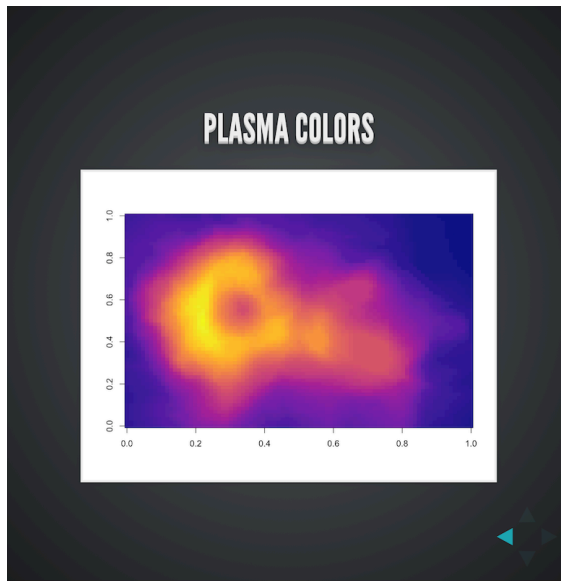


Figura 2: Screenshot di esempio preso da [QUI](#).

Struttura di un file R Markdown per slide:

```
---
title: "La Mia Analisi"
output: beamer_presentation
---

## Prima Slide

Testo e codice insieme:

{r}
summary(cars)

## Seconda Slide


$$E = mc^2$$

```

Punti di forza

- Integra codice R ed output nel documento
- Grafici generati automaticamente
- Output multipli: PDF, HTML, Word
- Diffusissimo in statistica e data science

Limiti

- Controllo del layout limitato
- Slide html dipendono dal browser
- Non ideale per tipografia fine
- Dipendenza da Pandoc

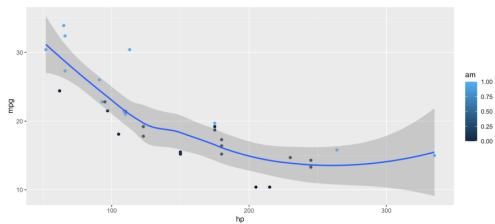
29.26.3. Quarto

Quarto è il successore ufficiale di R Markdown, sviluppato da Posit (ex RStudio). Supporta R, Python, Julia e Observable JS. Ha supporto **nativo** per Typst come formato di output dalla versione 1.4, il che lo rende un ponte naturale verso Typst.

6 / 29

MPG and Horsepower

```
1 library(ggplot2)
2 ggplot(mtcars, aes(hp, mpg, color = am)) +
3   geom_point() + geom_smooth(formula = y ~ x, method = "loess")
```



Learn more: [Executable Code](#)



Figura 3: Screenshot di esempio preso da [QUL](#).

Punti di forza

- Multi-linguaggio (R, Python, Julia)
- È il mio preferito (conta come punto di forza?)
- Output Typst nativo (v1.4+)
- Slide interattive con Reveal.js
- Ecosistema moderno e attivo (molto strutturato e la possibilità di far girare tanti linguaggi diversi è incredibile)

Limiti

- Astrazione che nasconde il controllo fine
- Template Typst limitati rispetto al nativo
- Curva di apprendimento per le opzioni

Configurazione Quarto con output Typst:

```
---
title: "Analisi Dati"
format:
  typst:
    toc: true
    papersize: a4
    mainfont: "Linux Libertine"
    section-numbering: "1.1."
---
```

Sezione

Testo e codice Python:

```
{python}
import numpy as np
print(np.mean([1, 2, 3, 4, 5]))
```

29.26.4. knitr

knitr è il motore di calcolo sottostante a R Markdown e Quarto. Esegue i blocchi di codice, cattura l'output (testo, grafici, tabelle) e lo inserisce nel documento. Con knitr puoi generare output Typst direttamente da R.

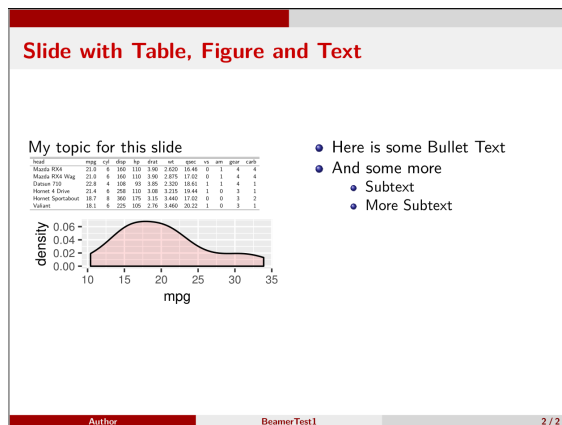


Figura 4: Screenshot di esempio preso da [QUI](#).

Uso di knitr con output Typst:

```
library(knitr)

# Imposta l'output Typst
opts_knit$set(out.format = "typst")

# Le tabelle vengono convertite automaticamente
knitr::kable(head(mtcars, 3), format = "pipe")

# Con tinytable (output Typst nativo)
library(tinytable)
tt(head(iris, 5)) |>
  style_tt(i = 0,
    background = "#1a3a5c",
    color = "white")
```

29.26.5. Polylux

Polylux è il pacchetto Typst più diffuso per le presentazioni. Aggiunge la gestione degli overlay (rivelazione progressiva degli elementi) e la modalità relatore al sistema di pagine di Typst.

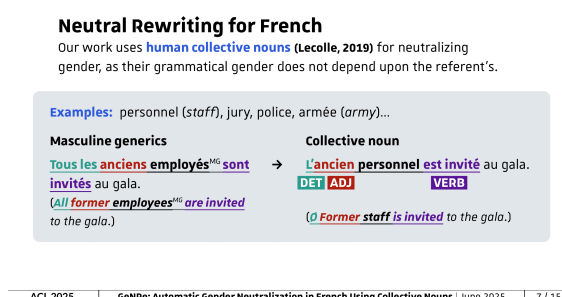


Figura 5: Screenshot di esempio preso da [QUI](#).

Punti di forza

- Integrazione totale con R
- Cache dei risultati computazionali
- Supporto output Typst (knitr ≥ 1.45)
- `tinytable`: tabelle R $\rightarrow$ Typst nativo

Limiti

- Solo per R (non Python)
- Richiede R installato
- Latenza di compilazione R

Punti di forza

- Sintassi Typst nativa
- Overlay e pause semplici
- Layout completamente libero
- Nessun tema predefinito da imparare

Limiti

- Temi predefiniti limitati
- Meno «magico» di Beamer

- Richiede più configurazione

Struttura base di una presentazione Polylux:

```
#import "@preview/polylux:0.3.1": *

#set page(paper: "presentation-16-9")
#set text(font: "Linux Libertine", size: 22pt)

#polylux-slide[
  = Titolo della Presentazione

  #align(center + horizon)[
    Sottotitolo descrittivo
    #v(1em)
    _Autore · Istituzione · Anno_
  ]
]

#polylux-slide[
  == Contenuto con rivelazione progressiva

  - Prima voce sempre visibile
  #pause
  - Seconda voce appare al click
  #pause
  - Terza voce appare dopo
]

#polylux-slide[
  == Due colonne

  #grid(columns: (1fr, 1fr), gutter: 2em,
    [*Colonna sinistra*\ Testo e contenuto],
    [*Colonna destra*\ Altro contenuto],
  )
]
```

29.26.6. Touying

Touying è il framework Typst più avanzato per presentazioni. Offre temi predefiniti professionali (Metropolis, University, Dewdrop, Stargazer...) e una gestione degli overlay molto sofisticata.

| Outline | |
|----------------------|---|
| Example Slides | 2 |
| Bullet Points | 3 |
| A CeTz Figure | 4 |
| References | 7 |

2025-10-08 1 / 8

Figura 6: Screenshot di esempio preso da [QUI](#).

Punti di forza

- Temi maturi e professionali
- Overlay avanzati (`#uncover` , `#only`)
- Handout mode automatico
- Barra di navigazione integrata

Limiti

- API più complessa di Polylux
- Versioni cambiano spesso (control-la Universe)
- Richiede più studio iniziale

Setup Touying con tema University:

```
#import "@preview/touying:0.5.3": *
#import themes.university: *

#show: university-theme.with(
  aspect-ratio: "16-9",
  config-info(
    title: [Il Titolo della Presentazione],
    subtitle: [Sottotitolo],
    author: [Mario Rossi],
    institution: [Università di Milano],
    date: datetime.today(),
  ),
)

// Slide titolo automatica
#title-slide()

// Sezione
= Prima Sezione

== Slide con overlay

#pause

Primo elemento — appare subito.

#pause

Secondo elemento — appare al secondo click.

== Slide con uncover

#uncover("2-") [Visibile dalla slide 2 in poi]
#only(3) [Visibile SOLO nella slide 3]
```

29.27. Confronto finale: quale scegliere?

| Strumento | Ideale per | Curva | Qualità PDF | Codice+Dati |
|------------|-----------------------------------|-------|-------------|-------------|
| Beamer | Articoli accademici con slide | Alta | ★★★★★ | No |
| R Markdown | Data science, statistica | Media | ★★★★☆ | R |
| Quarto | Multi-lingua, riproducibilità | Media | ★★★★☆ | R/Python |
| knitr | Engine sottostante a Quarto/Rmd | Media | ★★★★☆ | R |
| Polylux | Slide personalizzate in Typst | Bassa | ★★★★★ | No |
| Touying | Slide accademiche con temi pronti | Media | ★★★★★ | No |

Consiglio

Se già conosci LaTeX e vuoi la massima compatibilità con riviste e conferenze → **Beamer**. Se lavori con dati R/Python e vuoi riproducibilità computazionale → **Quarto** (con output Typst per la qualità). Se vuoi il controllo totale sul design e stai già usando Typst → **Touying** per temi pronti, **Polylux** per massima libertà.

In generale scegli quello che ti sembra il più semplice per quello che devi fare, io amo quarto: è bello, facile, veloce (ha un sacco di [Documentazione](#)) e la piattaforma [Posit Cloud](#) è fantastica.

Appendice F (Parte II) — Note Aggiuntive: Programmazione e Automazione

29.28. Il pacchetto Python `typst`

Il pacchetto ufficiale `typst` per Python consente di compilare documenti `.typ` direttamente da Python, senza usare il terminale o chiamate a `subprocess`.

```
pip install typst
```

29.28.1. Compilare un file

```
import typst

# Compila un file .typ esistente → PDF
typst.compile("documento.typ", output="documento.pdf")
```

```
# Compila da stringa Python → bytes PDF in memoria
sorgente = """
= Titolo generato da Python

Data odierna: #datetime.today().display()

- Prima voce
- Seconda voce
"""

pdf_bytes = typst.compile(sorgente)

with open("output.pdf", "wb") as f:
    f.write(pdf_bytes)
```

29.28.2. Passare parametri con sys.inputs

`sys.inputs` permette di iniettare valori stringa nel documento Typst dall'esterno, senza modificare il file sorgente:

```
import typst

typst.compile(
    "template.typ",
    output="report-mario.pdf",
    sys_inputs={
        "autore": "Mario Rossi",
        "titolo": "Report Trimestrale",
        "data": "Giugno 2026",
        "versione": "2.1",
    },
)
```

Nel file `template.typ`, leggi i valori così:

```
// I valori arrivano sempre come stringhe
#let autore = sys.inputs.at("autore", default: "Autore")
#let titolo = sys.inputs.at("titolo", default: "Documento")
#let data = sys.inputs.at("data", default: "")
#let versione = sys.inputs.at("versione", default: "1.0")

#set document(title: titolo, author: autore)

= #titolo

Redatto da *#autore* — #data — versione #versione
```

Da riga di comando il meccanismo equivalente è:

```
typst compile template.typ report.pdf \
--input autore="Mario Rossi" \
--input titolo="Report Trimestrale" \
--input data="Giugno 2026"
```

29.29. Leggere dati JSON nel documento

Per strutture dati complesse (tabelle, liste, metriche) è più pratico passare un file JSON che usare `sys.inputs`:

```
// Nel documento Typst: legge dati.json dalla stessa cartella
#let dati = json("dati.json")

// Accedi ai campi come un dizionario normale
= #dati.titolo

Autore: #dati.autore — Data: #dati.data

// Genera una tabella dai dati
#table(
  columns: dati.intestazioni.len(),
  stroke: 0.6pt + luma(200),
  fill: (_, row) => if row == 0 { C.blu } else { white },
  table.header(
    ..dati.intestazioni.map(h =>
      table.cell[#set text(fill: white, weight: "bold"); #h]
    ),
  ),
  ..dati.righe.map(r => r.map(c => [#c])).flatten(),
)
```

Il file `dati.json` corrispondente:

```
{
  "titolo": "Report Vendite — Q2 2026",
  "autore": "Team Analytics",
  "data": "30 giugno 2026",
  "intestazioni": ["Regione", "Fatturato", "Ordini", "Trend"],
  "righe": [
    ["Nord", "€ 58.100", "312", "↑ +8%"],
    ["Centro", "€ 41.200", "228", "↑ +15%"],
    ["Sud", "€ 43.000", "307", "↑ +13%"]
  ]
}
```

Da Python, genera il JSON e compila:

```
import typst, json
from datetime import date

dati = {
    "titolo": f"Report Vendite – Q2 {date.today().year}",
    "autore": "Team Analytics",
    "data": date.today().strftime("%d/%m/%Y"),
    "intestazioni": ["Regione", "Fatturato", "Ordini", "Trend"],
    "righe": [
        ["Nord", "€ 58.100", "312", "↑ +8%"],
        ["Centro", "€ 41.200", "228", "↑ +15%"],
        ["Sud", "€ 43.000", "307", "↑ +13%"],
    ],
}

with open("dati.json", "w", encoding="utf-8") as f:
    json.dump(dati, f, ensure_ascii=False, indent=2)

typst.compile("template.typ", output="report.pdf")
print("report.pdf generato!")
```

29.30. Il pattern Dati → Template → PDF

Il flusso fondamentale per la produzione automatica di documenti:



Casi d'uso reali per questo pattern:

- **Certificati personalizzati:** 500 certificati da un CSV con nomi e date — un solo template, un loop Python
- **Report settimanali:** dati da database → JSON → PDF inviato via email ogni lunedì mattina
- **Fatture automatiche:** ogni ordine del gestionale genera il suo PDF in pochi millisecondi
- **Dispense universitarie:** i docenti aggiornano i dati in un foglio Google → il PDF si rigenera da solo

29.31. Produzione in batch: esempio certificati

```
import typst, csv, os
```

```
# template-certificato.typ deve avere:
# #let nome = sys.inputs.at("nome", default: "")
# #let corso = sys.inputs.at("corso", default: "")
# #let data = sys.inputs.at("data", default: "")

os.makedirs("certificati", exist_ok=True)

with open("partecipanti.csv", newline="", encoding="utf-8") as f:
    for riga in csv.DictReader(f):
        typst.compile(
            "template-certificato.typ",
            output=f"certificati/{riga['id']}.pdf",
            sys_inputs={
                "nome": riga["nome"],
                "corso": riga["corso"],
                "data": riga["data"],
            },
        )
        print(f"Certificato per {riga['nome']} generato.")

print("Tutti i certificati sono pronti!")
```

29.32. Integrazione con FastAPI

Per generare PDF on-demand tramite un'API web:

```
from fastapi import FastAPI
from fastapi.responses import Response
import typst, json, tempfile, os

app = FastAPI()

@app.post("/pdf/report",
    response_class=Response,
    responses={200: {"content": {"application/pdf": {}}}})
async def genera_report(dati: dict):
    # Scrivi il JSON temporaneo
    with tempfile.NamedTemporaryFile(
        mode="w", suffix=".json",
        delete=False, encoding="utf-8") as f:
        json.dump(dati, f, ensure_ascii=False)
        json_path = f.name

    try:
        # Compila (typst legge il json dallo stesso path
        # solo se il template usa json("dati.json") con path assoluto)
        pdf = typst.compile("template-report.typ")
        return Response(
```

```
    content=pdf,
    media_type="application/pdf",
    headers={"Content-Disposition":
              "attachment; filename=report.pdf"},
  )
  finally:
    os.unlink(json_path)
```

29.33. GitHub Actions: report automatici

```
# .github/workflows/report.yml
name: Report Settimanale
on:
  schedule:
    - cron: "0 7 * * 1" # Ogni lunedì alle 7:00 UTC

jobs:
  genera:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with: { python-version: "3.12" }

      - name: Setup Typst
        uses: typst-community/setup-typst@v3

      - name: Installa dipendenze
        run: pip install typst pandas

      - name: Genera dati e PDF
        run: |
          python scripts/genera_dati.py # crea dati.json
          typst compile template.typ report.pdf

      - name: Carica artifact
        uses: actions/upload-artifact@v4
        with:
          name: report-settimanale
          path: report.pdf
```

29.34. Buone pratiche per Typst in produzione

| Pratica | Motivazione |
|---|---|
| Separa dati e template | Il <code>.typ</code> non cambia ad ogni esecuzione; i dati sì. Mantenerli separati semplifica manutenzione e debug. |
| Versiona i template con Git | Se un aggiornamento del template rompe i PDF, puoi tornare alla versione precedente in un comando. |
| Specifica la versione Typst | Aggiungi <code>typst.toml</code> al progetto con la versione esatta usata, per garantire riproducibilità nel tempo. |
| Testa con dati estremi | Tabelle con 0 righe, nomi lunghissimi, caratteri speciali (accenti, emoji): verifica che il template regga. |
| Font inclusi nel repository | Se usi font personalizzati, mettili nella cartella del progetto per evitare differenze tra macchine. |
| Usa <code>default:</code> in <code>sys.inputs</code> | Ogni <code>sys.inputs.at()</code> dovrebbe avere un valore di default per evitare crash se un parametro manca. |

Sei arrivato (finalmente) alla fine!

Ora hai tanti, se non tutti, gli strumenti per creare documenti professionali con Typst.

Il modo migliore per imparare è ovviamente **praticare** e **praticare**.

Spero che questo handbook vi sia stato utile.

Per qualsiasi cosa fatemi sapere sulla mia pagina contatti:

[STEFANO COELATI RAMA](#)